

Object-Oriented Programming (OOP) - 65+ Interview Questions with Answers

Comprehensive OOP Guide: From Fundamentals to Advanced Python Concepts

OOP Fundamentals (Questions 1-10)

Q1. What is Object-Oriented Programming?

Answer: Programming paradigm based on objects containing data (attributes) and code (methods).

Four pillars:

1. **Encapsulation:** Bundle data and methods
2. **Abstraction:** Hide complexity
3. **Inheritance:** Reuse code from parent classes
4. **Polymorphism:** One interface, multiple implementations

Benefits:

- Modularity
- Reusability
- Maintainability
- Scalability
- Real-world modeling

Q2. What is a class and an object?

Answer:

Class: Blueprint/template for objects

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"
```

Object: Instance of a class

```
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "Labrador")

print(dog1.bark()) # Buddy says Woof!
```

Analogy:

- Class = Cookie cutter
- Object = Cookie

Q3. Explain the four pillars of OOP in detail.

Answer:

1. Encapsulation: Bundling data and methods, hiding internal state.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

2. Abstraction: Hiding implementation details, showing only essential features.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        return "Car engine starts"
```

3. Inheritance: Child class inherits from parent class.

```
class Animal:
    def eat(self):
        return "Eating..."

class Dog(Animal):
    def bark(self):
        return "Woof!"

dog = Dog()
```

```
dog.eat()    # Inherited
dog.bark()   # Own method
```

4. Polymorphism: Same interface, different implementations.

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
```

Q4. What is encapsulation and why is it important?

Answer:

Definition: Wrapping data and methods together, controlling access.

Access modifiers (Python convention):

```
class Employee:
    def __init__(self):
        self.public = "accessible anywhere"
        self._protected = "convention: internal use"
        self.__private = "name mangling"

    def get_private(self):
        return self.__private
```

Benefits:

1. **Data hiding:** Protect internal state
2. **Flexibility:** Change implementation without affecting users
3. **Validation:** Control how data is modified
4. **Maintainability:** Clear interface

Example:

```
class Temperature:
    def __init__(self):
        self.__celsius = 0

    def set_celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self.__celsius = value

    def get_fahrenheit(self):
        return (self.__celsius * 9/5) + 32
```

Q5. Explain inheritance types with examples.

Answer:

1. Single Inheritance: One parent, one child.

```
class Parent:
    def parent_method(self):
        return "Parent"

class Child(Parent):
    def child_method(self):
        return "Child"
```

2. Multiple Inheritance: Multiple parents.

```
class Father:
    def skills(self):
        return "Gardening"

class Mother:
    def skills(self):
        return "Cooking"

class Child(Father, Mother):
    pass

child = Child()
child.skills() # MRO: Father's skills
```

3. Multilevel Inheritance: Chain of inheritance.

```
class Grandparent:
    pass
```

```
class Parent(Grandparent):  
    pass  
  
class Child(Parent):  
    pass
```

4. Hierarchical Inheritance: Multiple children from one parent.

```
class Animal:  
    pass  
  
class Dog(Animal):  
    pass  
  
class Cat(Animal):  
    pass
```

5. Hybrid Inheritance: Combination of above types.

Q6. What is polymorphism with examples?

Answer:

Types:

1. Method Overriding (Runtime):

```
class Animal:  
    def speak(self):  
        return "Some sound"  
  
class Dog(Animal):  
    def speak(self):  
        return "Woof!"  
  
class Cat(Animal):  
    def speak(self):  
        return "Meow!"  
  
animals = [Dog(), Cat()]  
for animal in animals:  
    print(animal.speak()) # Different implementations
```

2. Method Overloading (Compile-time, not in Python): Python doesn't support traditional overloading.

```
# Workaround with default args
def add(a, b=0, c=0):
    return a + b + c

add(1)          # 1
add(1, 2)       # 3
add(1, 2, 3)    # 6
```

3. Operator Overloading:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2  # Uses __add__
print(p3)    # (4, 6)
```

Duck Typing (Python):

```
def make_sound(animal):
    animal.speak()

# Works with any object having speak()
make_sound(Dog())
make_sound(Cat())
```

Q7. What is abstraction and how to implement it?

Answer:

Definition: Hide implementation details, show only functionality.

Abstract Base Class (ABC):

```
from abc import ABC, abstractmethod

class Shape(ABC):
```

```

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# shape = Shape() # Error: can't instantiate abstract class
rect = Rectangle(5, 10)
print(rect.area()) # 50

```

Benefits:

- Reduces complexity
- Code standardization
- Enforces contract
- Easier maintenance

Q8. Explain constructors and destructors.

Answer:

Constructor (`__init__`): Called when object is created.

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"Person {name} created")

person = Person("John", 30)

```

Types:**1. Default constructor:**

```
class MyClass:
    def __init__(self):
        self.value = 0
```

2. Parameterized constructor:

```
class MyClass:
    def __init__(self, value):
        self.value = value
```

Destructor (`__del__`): Called when object is destroyed.

```
class FileHandler:
    def __init__(self, filename):
        self.file = open(filename, 'w')

    def __del__(self):
        self.file.close()
        print("File closed")
```

Note: Python has garbage collection, `__del__` rarely needed.

Q9. What is `self` in Python?

Answer:

Definition: Reference to current instance.

Usage:

```
class Dog:
    def __init__(self, name):
        self.name = name # Instance variable

    def bark(self):
        return f"{self.name} says Woof!"

dog1 = Dog("Buddy")
dog2 = Dog("Max")

# self refers to dog1 in first call
print(dog1.bark()) # Buddy says Woof!

# self refers to dog2 in second call
print(dog2.bark()) # Max says Woof!
```

Behind the scenes:

```
dog1.bark()  
# Equivalent to:  
Dog.bark(dog1)
```

Why needed:

- Distinguish instance variables from local variables
- Access other methods
- Required by Python syntax

Q10. Explain class variables vs instance variables.

Answer:

Class variable: Shared by all instances.

```
class Dog:  
    species = "Canis familiaris" # Class variable  
  
    def __init__(self, name):  
        self.name = name # Instance variable  
  
dog1 = Dog("Buddy")  
dog2 = Dog("Max")  
  
print(dog1.species) # Canis familiaris  
print(dog2.species) # Canis familiaris  
  
# Changing class variable affects all  
Dog.species = "Dog"  
print(dog1.species) # Dog  
print(dog2.species) # Dog  
  
# Instance variables are unique  
print(dog1.name) # Buddy  
print(dog2.name) # Max
```

Access:

```
# Class variable  
ClassName.variable  
  
# Instance variable  
instance.variable
```

Advanced OOP Concepts (Questions 11-25)

Q11. What are static methods and class methods?

Answer:

Instance method (default):

```
class MyClass:
    def instance_method(self):
        return f"Instance: {self}"
```

Class method (@classmethod):

- First parameter is class (cls)
- Can access class variables
- Alternative constructors

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def from_string(cls, date_string):
        year, month, day = map(int, date_string.split('-'))
        return cls(year, month, day)

date = Date.from_string("2024-01-15")
```

Static method (@staticmethod):

- No self or cls parameter
- Utility functions
- Don't access instance or class state

```
class Math:
    @staticmethod
    def add(x, y):
        return x + y

result = Math.add(5, 3) # 8
```

When to use:

- **Instance:** Access instance data

- **Class:** Work with class data, factory methods
- **Static:** Utility functions, no access to class/instance

Q12. What is method overriding?

Answer:

Definition: Child class redefines parent's method.

Example:

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self): # Override
        return "Woof!"

class Cat(Animal):
    def speak(self): # Override
        return "Meow!"

dog = Dog()
print(dog.speak()) # Woof! (not "Some sound")
```

Calling parent method:

```
class Child(Parent):
    def method(self):
        # Call parent's method
        parent_result = super().method()
        # Add child's behavior
        return parent_result + " and more"
```

Rules:

- Same method name
- Same parameters (or compatible)
- Child's method replaces parent's

Q13. Explain super() function.

Answer:

Purpose: Access parent class methods.

Basic usage:

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call parent constructor
        self.age = age
```

Method calling:

```
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def greet(self):
        parent_greeting = super().greet()
        return parent_greeting + " and Child"
```

Multiple inheritance:

```
class A:
    def method(self):
        print("A")

class B(A):
    def method(self):
        print("B")
        super().method()

class C(A):
    def method(self):
        print("C")
        super().method()

class D(B, C):
    def method(self):
        print("D")
        super().method()

d = D()
d.method()
# Output: D, B, C, A (follows MRO)
```

MRO (Method Resolution Order):

```
print(D.__mro__)
```

Q14. What are magic/dunder methods?

Answer:

Definition: Special methods with double underscores.

Common magic methods:

Object creation:

```
__init__(self)      # Constructor
__new__(cls)         # Object creation
__del__(self)        # Destructor
```

String representation:

```
class Person:
    def __init__(self, name):
        self.name = name

    def __str__(self): # For users
        return f"Person: {self.name}"

    def __repr__(self): # For developers
        return f"Person('{self.name}')"

p = Person("John")
print(str(p))    # Person: John
print(repr(p))   # Person('John')
```

Arithmetic operators:

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __sub__(self, other):
        return Number(self.value - other.value)

    def __mul__(self, other):
        return Number(self.value * other.value)
```

Comparison:

```
__eq__(self, other)    # ==
__ne__(self, other)    # !=
__lt__(self, other)    # <
__le__(self, other)    # <=
__gt__(self, other)    # >
__ge__(self, other)    # >=
```

Container methods:

```
__len__(self)          # len()
__getitem__(self, key) # obj[key]
__setitem__(self, key, value) # obj[key] = value
__delitem__(self, key) # del obj[key]
__contains__(self, item) # item in obj
```

Q15. Explain composition vs inheritance.

Answer:**Inheritance** (IS-A relationship):

```
class Engine:
    def start(self):
        return "Engine started"

class Vehicle:
    pass

class Car(Vehicle): # Car IS-A Vehicle
    pass
```

Composition (HAS-A relationship):

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine() # Car HAS-AN Engine

    def start(self):
        return self.engine.start()
```

When to use:**Inheritance:**

- True IS-A relationship
- Code reuse from parent
- Polymorphism needed

Composition:

- HAS-A relationship
- More flexibility
- Avoid deep inheritance hierarchies
- Favor composition over inheritance (design principle)

Example:

```
# Bad: Inheritance abuse
class User(Database): # User is not a Database!
    pass

# Good: Composition
class User:
    def __init__(self, db):
        self.db = db # User HAS-A Database
```

Q16-25. [More advanced topics]

Q16. Explain Property decorators (@property).

Answer:

@property: Pythonic way to create getters, setters, and deleters without breaking encapsulation.

Basic usage:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        """Getter"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Setter with validation"""
        if value < -273.15:
```

```

        raise ValueError("Temperature below absolute zero!")
    self._celsius = value

    @celsius.deleter
    def celsius(self):
        """Deleter"""
        print("Deleting temperature")
        del self._celsius

    @property
    def fahrenheit(self):
        """Computed property"""
        return self._celsius * 9/5 + 32

# Usage
temp = Temperature(25)
print(temp.celsius)      # 25 (calls getter)
print(temp.fahrenheit)   # 77.0 (computed)

temp.celsius = 30        # Calls setter
print(temp.celsius)      # 30

# temp.celsius = -300    # ValueError!
del temp.celsius         # Calls deleter

```

Without @property (Java-style):

```

class Person:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, value):
        self._name = value

p = Person("John")
print(p.get_name()) # Verbose
p.set_name("Jane")

```

With @property (Pythonic):

```

class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):

```

```

        return self._name

    @name.setter
    def name(self, value):
        if not value:
            raise ValueError("Name cannot be empty")
        self._name = value

p = Person("John")
print(p.name)      # Clean!
p.name = "Jane"    # Looks like direct access, but validated

```

Read-only property:

```

class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):
        """Read-only computed property"""
        return 3.14159 * self._radius ** 2

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("Radius must be positive")
        self._radius = value

circle = Circle(5)
print(circle.area)      # 78.53975
circle.radius = 10
print(circle.area)      # 314.159

# circle.area = 100      # AttributeError: can't set attribute

```

Benefits:

- Maintains encapsulation
- Can add validation later without changing interface
- Computed properties
- Clean, Pythonic syntax

Q17. Explain Private, Protected, Public members.

Answer:

Python uses **naming conventions** for access control (no true private members).

Public (default):

```
class MyClass:
    def __init__(self):
        self.public = "Accessible anywhere"

    def public_method(self):
        return "Can be called from anywhere"

obj = MyClass()
print(obj.public)           # Works
print(obj.public_method())  # Works
```

Protected (single underscore _):

```
class Parent:
    def __init__(self):
        self._protected = "Convention: internal use only"

    def _protected_method(self):
        return "Should be used within class and subclasses"

class Child(Parent):
    def use_protected(self):
        return self._protected  # OK in subclass

obj = Parent()
print(obj._protected)  # Works, but convention says "don't do this"
```

Private (double underscore __ - Name Mangling):

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Name mangling applied

    def __validate(self, amount):  # Private method
        return amount > 0

    def deposit(self, amount):
        if self.__validate(amount):
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
```

```
print(account.get_balance()) # 1000

# Direct access fails
# print(account.__balance) # AttributeError

# But can still access via name mangling
print(account._BankAccount__balance) # 1000 (mangled name)
```

Name Mangling explained:

```
class MyClass:
    def __init__(self):
        self.__private = "Private"

    def show_attributes(self):
        print(dir(self))

obj = MyClass()
obj.show_attributes()
# You'll see: '_MyClass__private' instead of '__private'

# Accessing mangled name
print(obj._MyClass__private) # "Private"
```

Comparison:

Type	Convention	Accessible From	Name Mangling
Public	name	Anywhere	No
Protected	_name	Class & subclasses (convention)	No
Private	__name	Only within class	Yes

Best practices:

```
class GoodClass:
    def __init__(self):
        self.public_data = "Anyone can see"
        self._internal_data = "Use within class/subclasses"
        self.__truly_private = "Don't touch"

    def public_api(self):
        """Public interface"""
        return self._internal_helper()

    def _internal_helper(self):
        """Protected - can override in subclass"""
        return self.__secret_logic()
```

```
def __secret_logic(self):
    """Private - can't be overridden"""
    return "Secret"
```

Q18. Explain Multiple inheritance and Diamond problem.

Answer:

Multiple Inheritance: Class inherits from multiple parent classes.

Basic example:

```
class Father:
    def skills(self):
        return "Gardening"

class Mother:
    def skills(self):
        return "Cooking"

class Child(Father, Mother):
    pass

child = Child()
print(child.skills()) # "Gardening" (from Father, first in list)
```

Diamond Problem:



```
class A:
    def method(self):
        print("A's method")

class B(A):
    def method(self):
        print("B's method")

class C(A):
    def method(self):
        print("C's method")

class D(B, C):
```

```

    pass

d = D()
d.method() # Which method? B's or C's?
# Output: B's method (follows MRO)

```

MRO (Method Resolution Order) solves this:

```

print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>,
#  <class '__main__.C'>, <class '__main__.A'>,
#  <class 'object'>)

# Order: D → B → C → A → object
# Uses C3 Linearization algorithm

```

Practical example - Mixin pattern:

```

class LoggerMixin:
    def log(self, message):
        print(f"[LOG] {message}")

class TimestampMixin:
    def timestamp(self):
        from datetime import datetime
        return datetime.now().isoformat()

class User:
    def __init__(self, name):
        self.name = name

class AdminUser(LoggerMixin, TimestampMixin, User):
    def __init__(self, name):
        super().__init__(name)

    def perform_action(self, action):
        self.log(f"{self.name} {action} at {self.timestamp()}")

admin = AdminUser("Alice")
admin.perform_action("logged in")
# [LOG] Alice logged in at 2024-01-15T10:30:00

```

Using super() with multiple inheritance:

```

class A:
    def __init__(self):
        print("A init")

```

```

        super().__init__()

class B(A):
    def __init__(self):
        print("B init")
        super().__init__()

class C(A):
    def __init__(self):
        print("C init")
        super().__init__()

class D(B, C):
    def __init__(self):
        print("D init")
        super().__init__()

d = D()
# Output:
# D init
# B init
# C init
# A init
# (follows MRO)

```

Best practices:

- Prefer composition over multiple inheritance
- Use mixins for reusable behavior
- Keep inheritance hierarchies shallow
- Understand MRO before using multiple inheritance

Q19. Explain Method Resolution Order (MRO) and C3 Linearization.

Answer:

MRO: Order in which Python searches for methods in inheritance hierarchy.

C3 Linearization: Algorithm Python uses to determine MRO (since Python 2.3).

Rules:

1. Child comes before parents
2. Parents are searched in order they're listed
3. Each class appears only once
4. Preserves monotonicity (parent order consistent)

Simple inheritance:

```

class A:
    pass

```

```

class B(A):
    pass

class C(B):
    pass

print(C.__mro__)
# (<class 'C'>, <class 'B'>, <class 'A'>, <class 'object'>)
# Simple: C → B → A → object

```

Multiple inheritance:

```

class A:
    def method(self):
        print("A")

class B(A):
    def method(self):
        print("B")
        super().method()

class C(A):
    def method(self):
        print("C")
        super().method()

class D(B, C):
    def method(self):
        print("D")
        super().method()

print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>,
#  <class 'A'>, <class 'object'>)

d = D()
d.method()
# Output:
# D
# B
# C
# A

```

Why C before A? C3 ensures:

- D is before B and C (child before parents)
- B is before C (order in D(B, C))
- Both B and C are before A (they inherit from A)
- C3 linearization: D → B → C → A → object

Visualizing MRO:

```

import inspect

def show_mro(cls):
    print(f"\nMRO for {cls.__name__}:")
    for i, c in enumerate(cls.__mro__):
        print(f" {i}. {c.__name__}")

class Base1:
    def method(self):
        print("Base1")

class Base2:
    def method(self):
        print("Base2")

class Derived(Base1, Base2):
    pass

show_mro(Derived)
# MRO for Derived:
#  0. Derived
#  1. Base1
#  2. Base2
#  3. object

```

Complex diamond:

```

class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

class E(D, A): # Error or complex MRO?
    pass

# E's MRO will be calculated to avoid conflicts
print(E.__mro__)
# (<class 'E'>, <class 'D'>, <class 'B'>,
#  <class 'C'>, <class 'A'>, <class 'object'>)

```

Inconsistent hierarchy (Error):

```
class X:
    pass

class Y(X):
    pass

class Z(X, Y): # Error!
    pass

# TypeError: Cannot create a consistent method resolution
# order (MRO) for bases X, Y
# Reason: Y inherits from X, so X should come after Y,
# but Z(X, Y) says X should come before Y - contradiction!
```

Using MRO practically:

```
class LoggerMixin:
    def log(self, msg):
        print(f"[LOG] {msg}")

class ValidatorMixin:
    def validate(self):
        print("Validating...")
        return True

class Base:
    def save(self):
        print("Saving to database")

class Model(LoggerMixin, ValidatorMixin, Base):
    def save(self):
        self.log("Starting save")
        if self.validate():
            super().save() # Follows MRO
            self.log("Save complete")

model = Model()
model.save()
# [LOG] Starting save
# Validating...
# Saving to database
# [LOG] Save complete

print([c.__name__ for c in Model.__mro__])
# ['Model', 'LoggerMixin', 'ValidatorMixin', 'Base', 'object']
```

Key points:

- Use `super()` to respect MRO
- Check MRO with `Class.__mro__` or `Class.mro()`
- C3 prevents ambiguity in complex hierarchies
- MRO is left-to-right, depth-first (with C3 constraints)

Q20. Explain Abstract Base Classes (ABC module).

Answer:

ABC Module: Python's way to define abstract base classes and interfaces.

Basic usage:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    """Abstract base class"""

    @abstractmethod
    def area(self):
        """Must be implemented by subclasses"""
        pass

    @abstractmethod
    def perimeter(self):
        """Must be implemented by subclasses"""
        pass

    def description(self):
        """Concrete method - optional to override"""
        return f"Area: {self.area()}, Perimeter: {self.perimeter()}"

# Cannot instantiate abstract class
# shape = Shape() # TypeError: Can't instantiate abstract class

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

rect = Rectangle(10, 5)
print(rect.area())          # 50
print(rect.description())    # Area: 50, Perimeter: 30
```

Abstract properties:

```

from abc import ABC, abstractmethod

class Vehicle(ABC):
    @property
    @abstractmethod
    def max_speed(self):
        """Abstract property"""
        pass

    @abstractmethod
    def start_engine(self):
        pass

class Car(Vehicle):
    def __init__(self):
        self._max_speed = 200

    @property
    def max_speed(self):
        return self._max_speed

    def start_engine(self):
        return "Car engine started"

car = Car()
print(car.max_speed)      # 200
print(car.start_engine()) # Car engine started

```

ABCMeta metaclass (alternative syntax):

```

from abc import ABCMeta, abstractmethod

class Animal(metaclass=ABCMeta):
    @abstractmethod
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

```

Multiple abstract methods:

```

from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def connect(self):

```

```

        pass

    @abstractmethod
    def disconnect(self):
        pass

    @abstractmethod
    def query(self, sql):
        pass

    def execute_transaction(self, queries):
        """Concrete method using abstract methods"""
        self.connect()
        results = [self.query(q) for q in queries]
        self.disconnect()
        return results

class PostgreSQL(Database):
    def connect(self):
        return "Connected to PostgreSQL"

    def disconnect(self):
        return "Disconnected from PostgreSQL"

    def query(self, sql):
        return f"PostgreSQL query: {sql}"

db = PostgreSQL()
print(db.connect())
print(db.query("SELECT * FROM users"))

```

Partial implementation (still abstract):

```

from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def method1(self):
        pass

    @abstractmethod
    def method2(self):
        pass

class PartialImplementation(AbstractClass):
    def method1(self):
        return "Method 1 implemented"

    # method2 not implemented - still abstract

# Cannot instantiate
# obj = PartialImplementation() # TypeError

```

```
class CompleteImplementation(PartialImplementation):
    def method2(self):
        return "Method 2 implemented"

obj = CompleteImplementation() # Works!
```

Virtual subclasses (Register without inheritance):

```
from abc import ABC

class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass

class ThirdPartyShape:
    """Existing class we can't modify"""
    def draw(self):
        return "Drawing shape"

# Register as virtual subclass
Drawable.register(ThirdPartyShape)

shape = ThirdPartyShape()
print(isinstance(shape, Drawable)) # True
print(issubclass(ThirdPartyShape, Drawable)) # True
```

Benefits:

- Enforce interface contracts
- Document expected behavior
- Catch missing implementations at instantiation
- Type hinting support
- Virtual subclass registration

Q21. Explain Duck Typing and Dynamic Typing in Python.

Answer:

Duck Typing: "If it walks like a duck and quacks like a duck, it's a duck."

Philosophy: Focus on what an object can do, not what it is.

Example:

```
class Duck:
    def swim(self):
        return "Duck swimming"
```

```

    def quack(self):
        return "Quack!"

class Person:
    def swim(self):
        return "Person swimming"

    def quack(self):
        return "Person imitating duck: Quack!"

class Robot:
    def swim(self):
        return "Robot swimming"

    def quack(self):
        return "Beep boop quack"

# Function doesn't care about type, only behavior
def make_it_swim_and_quack(thing):
    print(thing.swim())
    print(thing.quack())

# Works with anything that has swim() and quack()
make_it_swim_and_quack(Duck())
make_it_swim_and_quack(Person())
make_it_swim_and_quack(Robot())

```

File-like objects (classic duck typing):

```

import io

def process_file(file_obj):
    """Works with any file-like object"""
    content = file_obj.read()
    return content.upper()

# Real file
with open('file.txt', 'r') as f:
    print(process_file(f))

# String buffer (duck typing!)
string_buffer = io.StringIO("hello world")
print(process_file(string_buffer))

# Custom file-like object
class CustomReader:
    def read(self):
        return "custom data"

print(process_file(CustomReader()))

```

Dynamic Typing: Variable types determined at runtime.

```
# Same variable, different types
x = 42          # int
print(type(x)) # <class 'int'>

x = "hello"     # str
print(type(x)) # <class 'str'>

x = [1, 2, 3]   # list
print(type(x)) # <class 'list'>

# Function can accept any type
def add(a, b):
    return a + b

print(add(5, 3))      # 8 (int)
print(add("Hi", "!")) # Hi! (str)
print(add([1], [2]))  # [1, 2] (list)
```

Duck typing with protocols:

```
# Works with anything iterable
def sum_all(items):
    total = 0
    for item in items:
        total += item
    return total

print(sum_all([1, 2, 3]))      # 6 (list)
print(sum_all((4, 5, 6)))     # 15 (tuple)
print(sum_all({7, 8, 9}))     # 24 (set)
print(sum_all(range(1, 4)))   # 6 (range)
```

EAFP vs LBYL:

EAFP (Easier to Ask for Forgiveness than Permission) - Pythonic:

```
def process_data(data):
    try:
        return data.process() # Duck typing
    except AttributeError:
        return "Cannot process"
```

LBYL (Look Before You Leap) - Not Pythonic:

```
def process_data(data):
    if hasattr(data, 'process'):
        return data.process()
    return "Cannot process"
```

Type hints (optional static typing):

```
from typing import Protocol

# Define protocol (duck typing with type checking)
class Drawable(Protocol):
    def draw(self) -> str:
        ...

class Circle:
    def draw(self) -> str:
        return "Drawing circle"

class Square:
    def draw(self) -> str:
        return "Drawing square"

def render(shape: Drawable):
    """Type checker knows shape must have draw()"""
    print(shape.draw())

render(Circle()) # OK
render(Square()) # OK
# render(42)     # Type checker error
```

Benefits:

- Flexible, reusable code
- No need for explicit interfaces
- Easier polymorphism
- Faster development

Drawbacks:

- Runtime errors if method missing
- Less explicit documentation
- IDE autocomplete limitations

Best practice:

```
from typing import Protocol, runtime_checkable

@runtime_checkable
```

```

class Swimmer(Protocol):
    def swim(self) -> str:
        ...

class Fish:
    def swim(self) -> str:
        return "Fish swimming"

fish = Fish()
print(isinstance(fish, Swimmer)) # True (runtime check)

```

Q22. Explain shallow copy vs deep copy.

Answer:

Shallow Copy: Creates new object but references same nested objects.

Deep Copy: Creates new object with copies of all nested objects.

Assignment (No copy):

```

original = [1, 2, [3, 4]]
reference = original # Just a reference

reference[0] = 99
print(original) # [99, 2, [3, 4]] - Changed!
print(reference is original) # True (same object)

```

Shallow copy:

```

import copy

original = [1, 2, [3, 4]]
shallow = copy.copy(original) # or original.copy() or original[:]

shallow[0] = 99
print(original) # [1, 2, [3, 4]] - Not changed
print(shallow) # [99, 2, [3, 4]]

# But nested objects are shared!
shallow[2][0] = 999
print(original) # [1, 2, [999, 4]] - CHANGED!
print(shallow) # [99, 2, [999, 4]]

```

Deep copy:

```
import copy

original = [1, 2, [3, 4]]
deep = copy.deepcopy(original)

deep[0] = 99
deep[2][0] = 999

print(original) # [1, 2, [3, 4]] - Not changed
print(deep)     # [99, 2, [999, 4]]
```

With objects:

```
import copy

class Person:
    def __init__(self, name, friends):
        self.name = name
        self.friends = friends # List of friends

person1 = Person("Alice", ["Bob", "Charlie"])

# Shallow copy
person2 = copy.copy(person1)
person2.name = "Alice Jr"
person2.friends.append("David")

print(person1.name) # Alice (not affected)
print(person1.friends) # ['Bob', 'Charlie', 'David'] (AFFECTED!)

# Deep copy
person3 = copy.deepcopy(person1)
person3.name = "Alice III"
person3.friends.append("Eve")

print(person1.name) # Alice
print(person1.friends) # ['Bob', 'Charlie', 'David'] (not affected)
```

Visualization:

```
import copy

# Original
original = {
    'name': 'John',
    'scores': [90, 85, 95],
    'address': {'city': 'NYC', 'zip': 10001}
}
```

```
# Shallow copy
shallow = copy.copy(original)
shallow['name'] = 'Jane'           # New value
shallow['scores'].append(100)      # Modifies original!
shallow['address']['city'] = 'LA'  # Modifies original!

print(original)
# {'name': 'John', 'scores': [90, 85, 95, 100],
#  'address': {'city': 'LA', 'zip': 10001}}

# Deep copy
deep = copy.deepcopy(original)
deep['name'] = 'Bob'
deep['scores'].append(75)
deep['address']['city'] = 'SF'

print(original)
# {'name': 'John', 'scores': [90, 85, 95, 100],
#  'address': {'city': 'LA', 'zip': 10001}} (not affected)
```

When to use:

- **Shallow:** When nested objects should be shared
- **Deep:** When you need complete independence

Q23. Explain dataclasses in Python.

Answer:

Dataclasses: Decorator to automatically generate special methods (since Python 3.7).

Basic usage:

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int
    city: str = "Unknown" # Default value

# Auto-generated __init__, __repr__, __eq__
person = Person("Alice", 30)
print(person) # Person(name='Alice', age=30, city='Unknown')

person2 = Person("Alice", 30)
print(person == person2) # True (auto __eq__)
```

Without dataclass (manual):

```

class PersonManual:
    def __init__(self, name, age, city="Unknown"):
        self.name = name
        self.age = age
        self.city = city

    def __repr__(self):
        return f"PersonManual(name='{self.name}', age={self.age},
city='{self.city}')"

    def __eq__(self, other):
        if not isinstance(other, PersonManual):
            return False
        return (self.name == other.name and
                self.age == other.age and
                self.city == other.city)

# Much more code for same result!

```

Features:

```

from dataclasses import dataclass, field
from typing import List

@dataclass(frozen=True) # Immutable
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
# p.x = 3.0 # FrozenInstanceError

@dataclass(order=True) # Comparison methods
class Score:
    value: int
    player: str = field(compare=False) # Exclude from comparison

s1 = Score(100, "Alice")
s2 = Score(90, "Bob")
print(s1 > s2) # True (compares value only)

@dataclass
class Team:
    name: str
    members: List[str] = field(default_factory=list) # Mutable default

team1 = Team("Team A")
team2 = Team("Team B")
team1.members.append("Alice")
print(team2.members) # [] (not shared!)

```

Post-init processing:

```

from dataclasses import dataclass

@dataclass
class Rectangle:
    width: float
    height: float
    area: float = field(init=False) # Computed, not in __init__

    def __post_init__(self):
        self.area = self.width * self.height

rect = Rectangle(10, 5)
print(rect.area) # 50

```

Benefits:

- Less boilerplate code
- Type hints included
- Auto-generated methods
- Immutability option (frozen=True)
- Default values and factories

Q24. Explain Python's special naming conventions summary.

Answer:**Single leading underscore `_var`:**

- Protected/internal use (convention)
- Not imported with `from module import *`

```

class MyClass:
    def _internal_method(self):
        pass

```

Single trailing underscore `var_`:

- Avoid naming conflicts with keywords

```

class_ = "MyClass" # Avoid conflict with 'class' keyword
type_ = "string"   # Avoid conflict with 'type'

```

Double leading underscore `__var`:

- Name mangling (strong suggestion of privacy)
- Renamed to `_ClassName__var`

```
class MyClass:
    def __init__(self):
        self.__private = "Hidden"

obj = MyClass()
# Access via mangled name:
print(obj._MyClass__private)
```

Double leading and trailing underscores `__var__`:

- Magic/dunder methods (special methods)
- Reserved by Python

```
__init__, __str__, __repr__, __add__, etc.
```

Single underscore `_`:

- Temporary/throwaway variable
- Last result in interpreter

```
for _ in range(5):
    print("Hello")

# Multiple assignment
_, y, _ = (1, 2, 3) # Only care about y
```

Summary table:

Pattern	Meaning	Example
<code>var</code>	Public	<code>name</code>
<code>_var</code>	Protected (convention)	<code>_internal</code>
<code>__var</code>	Private (name mangling)	<code>__secret</code>
<code>__var__</code>	Magic method	<code>__init__</code>
<code>var_</code>	Avoid keyword conflict	<code>class_</code>
<code>_</code>	Temporary/ignored	<code>for _ in range(5)</code>

Q25. Explain Python metaclasses.

Answer:

Metaclass: Class of a class. Defines how classes behave.

Everything is an object:

```
class MyClass:
    pass

obj = MyClass()

print(type(obj))      # <class '__main__.MyClass'>
print(type(MyClass))  # <class 'type'>
print(type(type))     # <class 'type'>

# type is a metaclass - creates classes
```

Creating classes dynamically with type:

```
# Normal way
class Dog:
    def bark(self):
        return "Woof!"

# Dynamic creation with type
Dog = type('Dog', (), {'bark': lambda self: "Woof!"})

dog = Dog()
print(dog.bark()) # Woof!
```

Custom metaclass:

```
class UpperAttrMetaclass(type):
    """Metaclass that uppercases all attribute names"""

    def __new__(cls, name, bases, attrs):
        uppercase_attrs = {
            (key.upper() if not key.startswith('__') else key): value
            for key, value in attrs.items()
        }
        return super().__new__(cls, name, bases, uppercase_attrs)

class MyClass(metaclass=UpperAttrMetaclass):
    x = 1
    y = 2

obj = MyClass()
print(obj.X) # 1 (was 'x')
print(obj.Y) # 2 (was 'y')
# print(obj.x) # AttributeError
```

Singleton metaclass:

```
class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Database(metaclass=SingletonMeta):
    def __init__(self):
        print("Connecting to database")

db1 = Database() # Connecting to database
db2 = Database() # (no output)
print(db1 is db2) # True
```

Validation metaclass:

```
class ValidatedMeta(type):
    def __new__(cls, name, bases, attrs):
        # Ensure class has required methods
        required_methods = ['validate', 'save']
        for method in required_methods:
            if method not in attrs:
                raise TypeError(f"Class {name} must implement {method}()")
        return super().__new__(cls, name, bases, attrs)

class User(metaclass=ValidatedMeta):
    def validate(self):
        return True

    def save(self):
        return "Saving user"

# class InvalidUser(metaclass=ValidatedMeta):
#     pass # TypeError: must implement validate() and save()
```

When to use:

- ORM frameworks (like Django models)
- API decorators
- Interface enforcement
- Class registration
- Singleton pattern

Warning: Metaclasses are powerful but complex. As Tim Peters said:

"Metaclasses are deeper magic than 99% of users should ever worry about."

Most use cases can be solved with:

- Decorators
- Class decorators
- `__init_subclass__`

Modern alternative (`__init_subclass__`):

```
class ValidatedBase:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        required = ['validate', 'save']
        for method in required:
            if not hasattr(cls, method):
                raise TypeError(f"Must implement {method}()")

class User(ValidatedBase):
    def validate(self):
        return True

    def save(self):
        return "Saving"

# Simpler than metaclass!
```

Design Patterns (Questions 26-40)

Q26. What is Singleton pattern?

Answer:

Purpose: Ensure only one instance of a class.

Implementation:

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
print(s1 is s2) # True (same object)
```

Using decorator:

```
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Database:
    def __init__(self):
        print("Connecting to database")

db1 = Database() # Connecting to database
db2 = Database() # (no output, reuses instance)
```

Use cases:

- Database connections
- Logging
- Configuration management
- Thread pools

Q27. What is Factory pattern?

Answer:

Purpose: Create objects without specifying exact class.

Implementation:

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Unknown animal type")
```

```
# Usage
animal = AnimalFactory.create_animal("dog")
print(animal.speak()) # Woof!
```

Benefits:

- Decoupling object creation
- Easy to add new types
- Centralized object creation logic

Q28. What is Observer pattern?

Answer:

Purpose: Notify multiple objects about state changes.

Implementation:

```
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

class Observer:
    def __init__(self, name):
        self.name = name

    def update(self, message):
        print(f"{self.name} received: {message}")

# Usage
subject = Subject()
obs1 = Observer("Observer 1")
obs2 = Observer("Observer 2")

subject.attach(obs1)
subject.attach(obs2)

subject.notify("Hello!")
# Observer 1 received: Hello!
# Observer 2 received: Hello!
```

Use cases:

- Event systems
- MVC architecture
- Publish-subscribe systems

Q29. Explain Strategy pattern.

Answer:

Strategy Pattern: Define a family of algorithms, encapsulate each one, and make them interchangeable.

Use case: When you have multiple ways to perform an operation and want to choose at runtime.

Example - Payment Processing:

```
from abc import ABC, abstractmethod

# Strategy interface
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

# Concrete strategies
class CreditCardPayment(PaymentStrategy):
    def __init__(self, card_number):
        self.card_number = card_number

    def pay(self, amount):
        return f"Paid ${amount} using Credit Card ending in {self.card_number[-4:]}]"

class PayPalPayment(PaymentStrategy):
    def __init__(self, email):
        self.email = email

    def pay(self, amount):
        return f"Paid ${amount} using PayPal account {self.email}"

class CryptoPayment(PaymentStrategy):
    def __init__(self, wallet_address):
        self.wallet_address = wallet_address

    def pay(self, amount):
        return f"Paid ${amount} using Crypto wallet {self.wallet_address[:10]}]..."

# Context
class ShoppingCart:
    def __init__(self):
        self.items = []
        self.payment_strategy = None
```

```

def add_item(self, item, price):
    self.items.append((item, price))

def set_payment_strategy(self, strategy):
    self.payment_strategy = strategy

def checkout(self):
    total = sum(price for item, price in self.items)
    return self.payment_strategy.pay(total)

# Usage
cart = ShoppingCart()
cart.add_item("Book", 29.99)
cart.add_item("Pen", 2.50)

# Choose payment method at runtime
cart.set_payment_strategy(CreditCardPayment("1234-5678-9012-3456"))
print(cart.checkout()) # Paid $32.49 using Credit Card

cart.set_payment_strategy(PayPalPayment("user@example.com"))
print(cart.checkout()) # Paid $32.49 using PayPal

```

Benefits:

- Easy to add new strategies
- Eliminates conditional statements
- Runtime algorithm selection

Q30. Explain Decorator pattern.

Answer:

Decorator Pattern: Add new functionality to objects dynamically without altering their structure.

Use case: Add features to objects at runtime (like adding toppings to pizza).

Example - Coffee Shop:

```

from abc import ABC, abstractmethod

# Component
class Coffee(ABC):
    @abstractmethod
    def cost(self):
        pass

    @abstractmethod
    def description(self):
        pass

# Concrete component

```

```
class SimpleCoffee(Coffee):
    def cost(self):
        return 5.0

    def description(self):
        return "Simple Coffee"

# Decorator base
class CoffeeDecorator(Coffee):
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost()

    def description(self):
        return self._coffee.description()

# Concrete decorators
class MilkDecorator(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 1.5

    def description(self):
        return self._coffee.description() + ", Milk"

class SugarDecorator(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 0.5

    def description(self):
        return self._coffee.description() + ", Sugar"

class WhippedCreamDecorator(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 2.0

    def description(self):
        return self._coffee.description() + ", Whipped Cream"

# Usage
coffee = SimpleCoffee()
print(f"{coffee.description():} ${coffee.cost()}")
# Simple Coffee: $5.0

coffee = MilkDecorator(coffee)
print(f"{coffee.description():} ${coffee.cost()}")
# Simple Coffee, Milk: $6.5

coffee = SugarDecorator(coffee)
print(f"{coffee.description():} ${coffee.cost()}")
# Simple Coffee, Milk, Sugar: $7.0

coffee = WhippedCreamDecorator(coffee)
```

```
print(f"{coffee.description():} ${coffee.cost()}")
# Simple Coffee, Milk, Sugar, Whipped Cream: $9.0
```

Python's built-in decorators:

```
def log_execution(func):
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@log_execution
def calculate(x, y):
    return x + y

calculate(5, 3)
# Executing calculate
# Finished calculate
```

Benefits:

- Add features without modifying original class
- Flexible alternative to subclassing
- Single Responsibility Principle

Q31. Explain Adapter pattern.

Answer:

Adapter Pattern: Convert interface of a class into another interface clients expect. Makes incompatible interfaces work together.

Use case: Integrate third-party libraries or legacy code with different interfaces.

Example - Media Player:

```
# Target interface (what client expects)
class MediaPlayer:
    def play(self, file_type, file_name):
        pass

# Adaptee (incompatible interface)
class AdvancedMediaPlayer:
    def play_mp4(self, file_name):
        print(f"Playing MP4 file: {file_name}")

    def play_mkv(self, file_name):
```

```

        print(f"Playing MKV file: {file_name}")

# Adapter
class MediaAdapter(MediaPlayer):
    def __init__(self):
        self.advanced_player = AdvancedMediaPlayer()

    def play(self, file_type, file_name):
        if file_type == "mp4":
            self.advanced_player.play_mp4(file_name)
        elif file_type == "mkv":
            self.advanced_player.play_mkv(file_name)
        else:
            print(f"Unsupported format: {file_type}")

# Concrete implementation
class AudioPlayer(MediaPlayer):
    def play(self, file_type, file_name):
        if file_type == "mp3":
            print(f"Playing MP3 file: {file_name}")
        elif file_type in ["mp4", "mkv"]:
            # Use adapter for advanced formats
            adapter = MediaAdapter()
            adapter.play(file_type, file_name)
        else:
            print(f"Invalid format: {file_type}")

# Usage
player = AudioPlayer()
player.play("mp3", "song.mp3")    # Playing MP3 file: song.mp3
player.play("mp4", "video.mp4")  # Playing MP4 file: video.mp4
player.play("mkv", "movie.mkv")  # Playing MKV file: movie.mkv
player.play("avi", "old.avi")    # Invalid format: avi

```

Real-world example:

```

# Adapting Celsius to Fahrenheit
class CelsiusTemperature:
    def get_temperature(self):
        return 25 # Celsius

class FahrenheitAdapter:
    def __init__(self, celsius_temp):
        self.celsius = celsius_temp

    def get_temperature(self):
        return (self.celsius.get_temperature() * 9/5) + 32

celsius = CelsiusTemperature()
fahrenheit = FahrenheitAdapter(celsius)
print(f"Temperature: {fahrenheit.get_temperature()}°F") # 77°F

```

Benefits:

- Reuse existing code
- Integrate incompatible interfaces
- Single Responsibility Principle

Q32. Explain Builder pattern.

Answer:

Builder Pattern: Construct complex objects step by step. Separate construction from representation.

Use case: When object creation involves many optional parameters or steps.

Example - Computer Builder:

```
class Computer:
    def __init__(self):
        self.cpu = None
        self.ram = None
        self.storage = None
        self.gpu = None
        self.os = None

    def __str__(self):
        return f"Computer(CPU: {self.cpu}, RAM: {self.ram}GB, " \
            f"Storage: {self.storage}GB, GPU: {self.gpu}, OS: {self.os})"

class ComputerBuilder:
    def __init__(self):
        self.computer = Computer()

    def set_cpu(self, cpu):
        self.computer.cpu = cpu
        return self # Return self for chaining

    def set_ram(self, ram):
        self.computer.ram = ram
        return self

    def set_storage(self, storage):
        self.computer.storage = storage
        return self

    def set_gpu(self, gpu):
        self.computer.gpu = gpu
        return self

    def set_os(self, os):
        self.computer.os = os
        return self
```

```

    def build(self):
        return self.computer

# Usage - Method chaining
gaming_pc = (ComputerBuilder()
    .set_cpu("Intel i9")
    .set_ram(32)
    .set_storage(1000)
    .set_gpu("RTX 4090")
    .set_os("Windows 11")
    .build())

print(gaming_pc)

office_pc = (ComputerBuilder()
    .set_cpu("Intel i5")
    .set_ram(16)
    .set_storage(512)
    .set_os("Windows 11")
    .build())

print(office_pc)

```

Director (optional):

```

class ComputerDirector:
    @staticmethod
    def build_gaming_pc(builder):
        return (builder
            .set_cpu("Intel i9")
            .set_ram(32)
            .set_storage(1000)
            .set_gpu("RTX 4090")
            .set_os("Windows 11")
            .build())

    @staticmethod
    def build_office_pc(builder):
        return (builder
            .set_cpu("Intel i5")
            .set_ram(8)
            .set_storage(256)
            .set_os("Windows 11")
            .build())

# Usage with director
director = ComputerDirector()
gaming = director.build_gaming_pc(ComputerBuilder())
office = director.build_office_pc(ComputerBuilder())

```

Benefits:

- Readable code for complex objects
- Avoids telescoping constructor
- Immutable objects possible
- Step-by-step construction

Q33. Explain Prototype pattern.

Answer:

Prototype Pattern: Create new objects by copying existing objects (cloning).

Use case: When object creation is expensive or complex, clone instead of creating from scratch.

Example - Document Cloning:

```
import copy

class Document:
    def __init__(self, title, content):
        self.title = title
        self.content = content
        self.images = []
        self.metadata = {}

    def add_image(self, image):
        self.images.append(image)

    def set_metadata(self, key, value):
        self.metadata[key] = value

    def clone(self):
        # Deep copy to avoid shared references
        return copy.deepcopy(self)

    def __str__(self):
        return f"Document(title='{self.title}', images={len(self.images)}, " \
               f"metadata={self.metadata})"

# Usage
original = Document("Report 2024", "This is the content")
original.add_image("chart1.png")
original.add_image("graph1.png")
original.set_metadata("author", "John Doe")
original.set_metadata("date", "2024-01-15")

print("Original:", original)

# Clone and modify
clone1 = original.clone()
clone1.title = "Report 2024 - Draft 2"
```

```

clone1.set_metadata("status", "draft")

print("Clone 1:", clone1)
print("Original unchanged:", original)

# Another clone
clone2 = original.clone()
clone2.title = "Report 2024 - Final"
clone2.add_image("summary.png")

print("Clone 2:", clone2)

```

Prototype Registry:

```

class PrototypeRegistry:
    def __init__(self):
        self._prototypes = {}

    def register(self, name, prototype):
        self._prototypes[name] = prototype

    def unregister(self, name):
        del self._prototypes[name]

    def clone(self, name):
        return copy.deepcopy(self._prototypes[name])

# Usage
registry = PrototypeRegistry()

# Register prototypes
basic_doc = Document("Basic Template", "")
registry.register("basic", basic_doc)

report_doc = Document("Report Template", "Executive Summary:\n\n")
report_doc.set_metadata("type", "report")
registry.register("report", report_doc)

# Clone from registry
my_doc = registry.clone("report")
my_doc.title = "Q1 Sales Report"

```

Benefits:

- Avoid expensive object creation
- Reduce subclassing
- Dynamic object creation

Q34. Explain Facade pattern.

Answer:

Facade Pattern: Provide simplified interface to complex subsystem.

Use case: Hide complexity of multiple classes behind single interface.

Example - Home Theater System:

```
# Complex subsystem classes
class Amplifier:
    def on(self):
        print("Amplifier: Turning on")

    def set_volume(self, level):
        print(f"Amplifier: Setting volume to {level}")

    def off(self):
        print("Amplifier: Turning off")

class DVDPlayer:
    def on(self):
        print("DVD Player: Turning on")

    def play(self, movie):
        print(f"DVD Player: Playing '{movie}'")

    def stop(self):
        print("DVD Player: Stopping")

    def off(self):
        print("DVD Player: Turning off")

class Projector:
    def on(self):
        print("Projector: Turning on")

    def set_input(self, source):
        print(f"Projector: Setting input to {source}")

    def off(self):
        print("Projector: Turning off")

class Lights:
    def dim(self, level):
        print(f"Lights: Dimming to {level}%")

    def on(self):
        print("Lights: Turning on")

# Facade - Simplified interface
class HomeTheaterFacade:
    def __init__(self):
        self.amplifier = Amplifier()
```

```

        self.dvd = DVDPlayer()
        self.projector = Projector()
        self.lights = Lights()

    def watch_movie(self, movie):
        print("\n=== Starting Movie ===")
        self.lights.dim(10)
        self.projector.on()
        self.projector.set_input("DVD")
        self.amplifier.on()
        self.amplifier.set_volume(5)
        self.dvd.on()
        self.dvd.play(movie)
        print("=== Enjoy! ===\n")

    def end_movie(self):
        print("\n=== Ending Movie ===")
        self.dvd.stop()
        self.dvd.off()
        self.amplifier.off()
        self.projector.off()
        self.lights.on()
        print("=== Shutdown Complete ===\n")

# Usage - Simple!
theater = HomeTheaterFacade()
theater.watch_movie("The Matrix")
# ... watch movie ...
theater.end_movie()

```

Without Facade (complex):

```

# User has to know all subsystems
amp = Amplifier()
dvd = DVDPlayer()
proj = Projector()
lights = Lights()

lights.dim(10)
proj.on()
proj.set_input("DVD")
amp.on()
amp.set_volume(5)
dvd.on()
dvd.play("The Matrix")
# ... complex!

```

Benefits:

- Simplifies complex systems

- Loose coupling
- Easier to use and understand

Q35. Explain Proxy pattern.

Answer:

Proxy Pattern: Provide placeholder/surrogate for another object to control access.

Types: Virtual Proxy, Protection Proxy, Remote Proxy, Caching Proxy

Example - Image Proxy (Lazy Loading):

```
from abc import ABC, abstractmethod

# Subject interface
class Image(ABC):
    @abstractmethod
    def display(self):
        pass

# Real subject
class RealImage(Image):
    def __init__(self, filename):
        self.filename = filename
        self.load_from_disk()

    def load_from_disk(self):
        print(f"Loading image from disk: {self.filename}")
        # Expensive operation

    def display(self):
        print(f"Displaying: {self.filename}")

# Proxy - Delays loading until needed
class ImageProxy(Image):
    def __init__(self, filename):
        self.filename = filename
        self.real_image = None

    def display(self):
        if self.real_image is None:
            self.real_image = RealImage(self.filename)
            self.real_image.display()

# Usage
print("Creating image proxies...")
image1 = ImageProxy("photo1.jpg")
image2 = ImageProxy("photo2.jpg")
print("Proxies created (images not loaded yet)\n")

print("Displaying image1 for first time:")
image1.display() # Loads from disk
```

```
print()

print("Displaying image1 again:")
image1.display() # Already loaded, just displays
print()

print("Displaying image2:")
image2.display() # Loads from disk
```

Protection Proxy (Access Control):

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return f"Withdrew ${amount}. New balance: ${self.balance}"
        return "Insufficient funds"

class BankAccountProxy:
    def __init__(self, account, user_role):
        self.account = account
        self.user_role = user_role

    def withdraw(self, amount):
        if self.user_role == "owner":
            return self.account.withdraw(amount)
        elif self.user_role == "viewer":
            return "Access denied: Viewers cannot withdraw"
        else:
            return "Access denied: Invalid role"

    def get_balance(self):
        return f"Balance: ${self.account.balance}"

# Usage
account = BankAccount(1000)

owner_proxy = BankAccountProxy(account, "owner")
print(owner_proxy.withdraw(100)) # Allowed

viewer_proxy = BankAccountProxy(account, "viewer")
print(viewer_proxy.withdraw(100)) # Denied
print(viewer_proxy.get_balance()) # Allowed
```

Caching Proxy:

```

class DatabaseQuery:
    def query(self, sql):
        print(f"Executing SQL: {sql}")
        # Expensive database operation
        return f"Results for: {sql}"

class CachingQueryProxy:
    def __init__(self):
        self.db = DatabaseQuery()
        self.cache = {}

    def query(self, sql):
        if sql in self.cache:
            print(f"Returning cached result for: {sql}")
            return self.cache[sql]

        result = self.db.query(sql)
        self.cache[sql] = result
        return result

# Usage
proxy = CachingQueryProxy()
proxy.query("SELECT * FROM users") # Executes
proxy.query("SELECT * FROM users") # From cache

```

Benefits:

- Lazy initialization
- Access control
- Caching
- Logging/monitoring

Q36. Explain Command pattern.

Answer:

Command Pattern: Encapsulate request as object, allowing parameterization and queuing of requests.

Use case: Undo/redo, task queuing, macro recording.

Example - Text Editor:

```

from abc import ABC, abstractmethod

# Command interface
class Command(ABC):
    @abstractmethod
    def execute(self):
        pass

    @abstractmethod

```

```
def undo(self):
    pass

# Receiver
class TextEditor:
    def __init__(self):
        self.text = ""

    def write(self, text):
        self.text += text
        print(f"Current text: '{self.text}'")

    def delete(self, length):
        deleted = self.text[-length:] if length <= len(self.text) else self.text
        self.text = self.text[:-length] if length <= len(self.text) else ""
        print(f"Current text: '{self.text}'")
        return deleted

# Concrete commands
class WriteCommand(Command):
    def __init__(self, editor, text):
        self.editor = editor
        self.text = text

    def execute(self):
        self.editor.write(self.text)

    def undo(self):
        self.editor.delete(len(self.text))

class DeleteCommand(Command):
    def __init__(self, editor, length):
        self.editor = editor
        self.length = length
        self.deleted_text = None

    def execute(self):
        self.deleted_text = self.editor.delete(self.length)

    def undo(self):
        if self.deleted_text:
            self.editor.write(self.deleted_text)

# Invoker
class CommandHistory:
    def __init__(self):
        self.history = []

    def execute(self, command):
        command.execute()
        self.history.append(command)

    def undo(self):
        if self.history:
```

```

        command = self.history.pop()
        command.undo()
    else:
        print("Nothing to undo")

# Usage
editor = TextEditor()
history = CommandHistory()

history.execute(WriteCommand(editor, "Hello "))
history.execute(WriteCommand(editor, "World"))
history.execute>DeleteCommand(editor, 5)) # Delete "World"

print("\nUndo operations:")
history.undo() # Restore "World"
history.undo() # Remove "World"
history.undo() # Remove "Hello "
```

Remote Control Example:

```

# Receiver classes
class Light:
    def on(self):
        print("Light is ON")

    def off(self):
        print("Light is OFF")

class Fan:
    def high(self):
        print("Fan on HIGH")

    def off(self):
        print("Fan is OFF")

# Commands
class LightOnCommand(Command):
    def __init__(self, light):
        self.light = light

    def execute(self):
        self.light.on()

    def undo(self):
        self.light.off()

class FanHighCommand(Command):
    def __init__(self, fan):
        self.fan = fan

    def execute(self):
        self.fan.high()
```

```

    def undo(self):
        self.fan.off()

# Remote control (invoker)
class RemoteControl:
    def __init__(self):
        self.command = None

    def set_command(self, command):
        self.command = command

    def press_button(self):
        self.command.execute()

    def press_undo(self):
        self.command.undo()

# Usage
remote = RemoteControl()
light = Light()
fan = Fan()

remote.set_command(LightOnCommand(light))
remote.press_button() # Light is ON
remote.press_undo()   # Light is OFF

remote.set_command(FanHighCommand(fan))
remote.press_button() # Fan on HIGH

```

Benefits:

- Decouples sender and receiver
- Undo/redo support
- Macro commands (batch)
- Queue and log requests

Q37. Explain State pattern.

Answer:

State Pattern: Object behavior changes when internal state changes. Object appears to change class.

Use case: Finite state machines, workflow systems.

Example - Vending Machine:

```

from abc import ABC, abstractmethod

# State interface
class State(ABC):

```

```
@abstractmethod
def insert_coin(self, machine):
    pass

@abstractmethod
def select_product(self, machine):
    pass

@abstractmethod
def dispense(self, machine):
    pass

# Concrete states
class NoCoinState(State):
    def insert_coin(self, machine):
        print("Coin inserted")
        machine.set_state(machine.has_coin_state)

    def select_product(self, machine):
        print("Insert coin first")

    def dispense(self, machine):
        print("Insert coin first")

class HasCoinState(State):
    def insert_coin(self, machine):
        print("Coin already inserted")

    def select_product(self, machine):
        print("Product selected")
        machine.set_state(machine.dispensing_state)

    def dispense(self, machine):
        print("Select product first")

class DispensingState(State):
    def insert_coin(self, machine):
        print("Please wait, dispensing product")

    def select_product(self, machine):
        print("Please wait, dispensing product")

    def dispense(self, machine):
        print("Dispensing product...")
        machine.count -= 1
        if machine.count > 0:
            machine.set_state(machine.no_coin_state)
        else:
            print("Out of stock")
            machine.set_state(machine.out_of_stock_state)

class OutOfStockState(State):
    def insert_coin(self, machine):
        print("Out of stock, returning coin")
```

```
def select_product(self, machine):
    print("Out of stock")

def dispense(self, machine):
    print("Out of stock")

# Context
class VendingMachine:
    def __init__(self, count):
        self.count = count

        # Create state objects
        self.no_coin_state = NoCoinState()
        self.has_coin_state = HasCoinState()
        self.dispensing_state = DispensingState()
        self.out_of_stock_state = OutOfStockState()

        # Initial state
        self.state = self.no_coin_state if count > 0 else self.out_of_stock_state

    def set_state(self, state):
        self.state = state

    def insert_coin(self):
        self.state.insert_coin(self)

    def select_product(self):
        self.state.select_product(self)

    def dispense(self):
        self.state.dispense(self)

# Usage
machine = VendingMachine(2)

print("=== Transaction 1 ===")
machine.insert_coin()
machine.select_product()
machine.dispense()

print("\n=== Transaction 2 ===")
machine.insert_coin()
machine.select_product()
machine.dispense()

print("\n=== Transaction 3 (out of stock) ===")
machine.insert_coin()
```

Benefits:

- Eliminates conditional statements

- Each state in separate class
- Easy to add new states
- Clear state transitions

Q38. Explain Template Method pattern.

Answer:

Template Method Pattern: Define skeleton of algorithm, let subclasses override specific steps.

Use case: When algorithm steps are same but implementation differs.

Example - Data Processing:

```
from abc import ABC, abstractmethod

class DataProcessor(ABC):
    # Template method
    def process(self):
        self.read_data()
        self.process_data()
        self.save_data()
        self.cleanup()

    @abstractmethod
    def read_data(self):
        pass

    @abstractmethod
    def process_data(self):
        pass

    @abstractmethod
    def save_data(self):
        pass

    # Hook method (optional override)
    def cleanup(self):
        print("Default cleanup")

# Concrete implementations
class CSVDataProcessor(DataProcessor):
    def read_data(self):
        print("Reading CSV file")
        self.data = ["csv", "data"]

    def process_data(self):
        print(f"Processing CSV: {self.data}")
        self.result = [x.upper() for x in self.data]

    def save_data(self):
        print(f"Saving to CSV: {self.result}")
```

```

class JSONDataProcessor(DataProcessor):
    def read_data(self):
        print("Reading JSON file")
        self.data = {"key": "json", "value": "data"}

    def process_data(self):
        print(f"Processing JSON: {self.data}")
        self.result = {k: v.upper() for k, v in self.data.items()}

    def save_data(self):
        print(f"Saving to JSON: {self.result}")

    def cleanup(self):
        print("Custom JSON cleanup")

# Usage
print("=== CSV Processing ===")
csv_processor = CSVDataProcessor()
csv_processor.process()

print("\n=== JSON Processing ===")
json_processor = JSONDataProcessor()
json_processor.process()

```

Example - Game Framework:

```

class Game(ABC):
    # Template method
    def play(self):
        self.initialize()
        self.start_play()
        self.end_play()

    @abstractmethod
    def initialize(self):
        pass

    @abstractmethod
    def start_play(self):
        pass

    @abstractmethod
    def end_play(self):
        pass

class Chess(Game):
    def initialize(self):
        print("Chess: Setting up board")

    def start_play(self):
        print("Chess: Game started")

```

```

    def end_play(self):
        print("Chess: Winner declared")

class Soccer(Game):
    def initialize(self):
        print("Soccer: Setting up field")

    def start_play(self):
        print("Soccer: Kickoff!")

    def end_play(self):
        print("Soccer: Final whistle")

# Usage
chess = Chess()
chess.play()

print()

soccer = Soccer()
soccer.play()

```

Benefits:

- Code reuse
- Control over algorithm flow
- Consistent structure
- Hollywood Principle ("Don't call us, we'll call you")

Q39. Explain Dependency Injection.

Answer:

Dependency Injection: Provide dependencies from outside rather than creating them internally.

Types: Constructor Injection, Setter Injection, Interface Injection

Without DI (Tight Coupling):

```

class MySQLDatabase:
    def query(self, sql):
        return f"MySQL: {sql}"

class UserService:
    def __init__(self):
        self.db = MySQLDatabase() # Hard-coded dependency

    def get_user(self, user_id):
        return self.db.query(f"SELECT * FROM users WHERE id={user_id}")

```

```
# Problem: Can't easily switch to PostgreSQL or test with mock
```

With DI (Loose Coupling):

```
from abc import ABC, abstractmethod

# Abstraction
class Database(ABC):
    @abstractmethod
    def query(self, sql):
        pass

# Implementations
class MySQLDatabase(Database):
    def query(self, sql):
        return f"MySQL: {sql}"

class PostgreSQLDatabase(Database):
    def query(self, sql):
        return f"PostgreSQL: {sql}"

class MockDatabase(Database):
    def query(self, sql):
        return f"Mock: {sql}"

# Service with DI
class UserService:
    def __init__(self, database: Database):
        self.db = database # Injected dependency

    def get_user(self, user_id):
        return self.db.query(f"SELECT * FROM users WHERE id={user_id}")

# Usage - Flexibility!
mysql_service = UserService(MySQLDatabase())
print(mysql_service.get_user(1))

postgres_service = UserService(PostgreSQLDatabase())
print(postgres_service.get_user(1))

# Easy testing
mock_service = UserService(MockDatabase())
print(mock_service.get_user(1))
```

Constructor Injection (most common):

```
class EmailService:
    def send(self, to, message):
```

```
        print(f"Sending email to {to}: {message}")

class NotificationService:
    def __init__(self, email_service: EmailService):
        self.email_service = email_service # Constructor injection

    def notify(self, user, message):
        self.email_service.send(user, message)

email = EmailService()
notifier = NotificationService(email)
notifier.notify("user@example.com", "Hello!")
```

Setter Injection:

```
class Logger:
    def log(self, message):
        print(f"LOG: {message}")

class Application:
    def __init__(self):
        self.logger = None

    def set_logger(self, logger: Logger):
        self.logger = logger # Setter injection

    def run(self):
        if self.logger:
            self.logger.log("Application started")

app = Application()
app.set_logger(Logger())
app.run()
```

DI Container (Framework):

```
class DIContainer:
    def __init__(self):
        self._services = {}

    def register(self, interface, implementation):
        self._services[interface] = implementation

    def resolve(self, interface):
        return self._services[interface]()

# Usage
container = DIContainer()
container.register('database', MySQLDatabase)
```

```
container.register('user_service', lambda:
UserService(container.resolve('database')))

service = container.resolve('user_service')
print(service.get_user(1))
```

Benefits:

- Loose coupling
- Easy testing (mock dependencies)
- Flexibility (swap implementations)
- Follows Dependency Inversion Principle

Q40. Explain MVC pattern.

Answer:

MVC Pattern: Separate application into Model, View, Controller.

Components:

- **Model:** Data and business logic
- **View:** UI presentation
- **Controller:** Handles user input, updates model and view

Example - Todo App:

```
# Model
class TodoModel:
    def __init__(self):
        self.todos = []

    def add_todo(self, task):
        todo = {"id": len(self.todos) + 1, "task": task, "done": False}
        self.todos.append(todo)
        return todo

    def get_todos(self):
        return self.todos

    def mark_done(self, todo_id):
        for todo in self.todos:
            if todo["id"] == todo_id:
                todo["done"] = True
                return True
        return False

    def delete_todo(self, todo_id):
        self.todos = [t for t in self.todos if t["id"] != todo_id]

# View
```

```

class TodoView:
    def display_todos(self, todos):
        print("\n=== Todo List ===")
        if not todos:
            print("No todos!")
        for todo in todos:
            status = "✓" if todo["done"] else "o"
            print(f"{status} [{todo['id']}] {todo['task']}")
        print("=" * 20)

    def display_message(self, message):
        print(f"\n>>> {message}")

    def get_user_input(self, prompt):
        return input(prompt)

# Controller
class TodoController:
    def __init__(self, model, view):
        self.model = model
        self.view = view

    def add_todo(self, task):
        todo = self.model.add_todo(task)
        self.view.display_message(f"Added: {task}")
        self.show_todos()

    def show_todos(self):
        todos = self.model.get_todos()
        self.view.display_todos(todos)

    def mark_done(self, todo_id):
        if self.model.mark_done(todo_id):
            self.view.display_message(f"Marked todo {todo_id} as done")
        else:
            self.view.display_message(f"Todo {todo_id} not found")
        self.show_todos()

    def delete_todo(self, todo_id):
        self.model.delete_todo(todo_id)
        self.view.display_message(f"Deleted todo {todo_id}")
        self.show_todos()

    def run(self):
        while True:
            self.show_todos()
            print("\nCommands: (a)dd, (d)one, (r)emove, (q)uit")
            command = self.view.get_user_input("Enter command: ").lower()

            if command == 'a':
                task = self.view.get_user_input("Enter task: ")
                self.add_todo(task)
            elif command == 'd':
                todo_id = int(self.view.get_user_input("Enter todo ID: "))

```

```

        self.mark_done(todo_id)
    elif command == 'r':
        todo_id = int(self.view.get_user_input("Enter todo ID: "))
        self.delete_todo(todo_id)
    elif command == 'q':
        break

# Usage
if __name__ == "__main__":
    model = TodoModel()
    view = TodoView()
    controller = TodoController(model, view)

    # Pre-populate for demo
    controller.add_todo("Buy groceries")
    controller.add_todo("Write code")
    controller.add_todo("Exercise")

    # Uncomment to run interactive
    # controller.run()

```

Web MVC Example:

```

# Model
class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email

class UserModel:
    def __init__(self):
        self.users = {}

    def create_user(self, username, email):
        user = User(username, email)
        self.users[username] = user
        return user

    def get_user(self, username):
        return self.users.get(username)

# View (Template)
class UserView:
    def render_user(self, user):
        return f"""
        <html>
            <body>
                <h1>User Profile</h1>
                <p>Username: {user.username}</p>
                <p>Email: {user.email}</p>
            </body>
        </html>

```

```

"""

def render_user_list(self, users):
    user_list = "".join([f"<li>{u.username}</li>" for u in users])
    return f"""
    <html>
        <body>
            <h1>All Users</h1>
            <ul>{user_list}</ul>
        </body>
    </html>
    """

# Controller
class UserController:
    def __init__(self, model, view):
        self.model = model
        self.view = view

    def create_user(self, username, email):
        user = self.model.create_user(username, email)
        return self.view.render_user(user)

    def show_user(self, username):
        user = self.model.get_user(username)
        if user:
            return self.view.render_user(user)
        return "<html><body><h1>User not found</h1></body></html>"

# Usage
model = UserModel()
view = UserView()
controller = UserController(model, view)

# Simulate web requests
html = controller.create_user("john", "john@example.com")
print(html)

html = controller.show_user("john")
print(html)

```

Benefits:

- Separation of concerns
- Easier testing (test each component independently)
- Multiple views for same model
- Parallel development
- Code reusability

Variants:

- **MVP** (Model-View-Presenter): View is passive

- **MVVM** (Model-View-ViewModel): Two-way data binding
- **MVT** (Model-View-Template): Django's approach

SOLID Principles (Questions 41-50)

Q41. What are SOLID principles?

Answer:

S - Single Responsibility Principle **O** - Open/Closed Principle **L** - Liskov Substitution Principle **I** - Interface Segregation Principle **D** - Dependency Inversion Principle

Q42. Explain Single Responsibility Principle (SRP).

Answer:

Principle: A class should have only one reason to change.

Bad example:

```
class User:
    def __init__(self, name):
        self.name = name

    def get_user_data(self):
        pass

    def save_to_database(self): # Database responsibility
        pass

    def send_email(self): # Email responsibility
        pass
```

Good example:

```
class User:
    def __init__(self, name):
        self.name = name

class UserRepository:
    def save(self, user):
        # Database logic
        pass

class EmailService:
    def send_email(self, user):
        # Email logic
        pass
```

Q43. Explain Open/Closed Principle (OCP).

Answer:

Principle: Open for extension, closed for modification.

Bad example:

```
class AreaCalculator:
    def calculate(self, shape):
        if shape.type == "circle":
            return 3.14 * shape.radius ** 2
        elif shape.type == "rectangle":
            return shape.width * shape.height
        # Need to modify this method for new shapes
```

Good example:

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

# Can add new shapes without modifying existing code
class Triangle(Shape):
    def area(self):
        # Implementation
        pass
```

Q44. Explain Liskov Substitution Principle (LSP).

Answer:

Principle: Subtypes must be substitutable for their base types.

Bad example:

```
class Bird:
    def fly(self):
        return "Flying"

class Penguin(Bird):
    def fly(self):
        raise Exception("Can't fly!") # Violates LSP
```

Good example:

```
class Bird:
    pass

class FlyingBird(Bird):
    def fly(self):
        return "Flying"

class Penguin(Bird):
    def swim(self):
        return "Swimming"

class Sparrow(FlyingBird):
    pass
```

Q45. Explain Interface Segregation Principle (ISP).

Answer:

Principle: Don't force classes to depend on methods they don't use.

Bad example:

```
class Worker(ABC):
    @abstractmethod
    def work(self):
        pass

    @abstractmethod
    def eat(self):
        pass

class Robot(Worker):
    def work(self):
        return "Working"
```

```
def eat(self):  
    pass # Robots don't eat!
```

Good example:

```
class Workable(ABC):  
    @abstractmethod  
    def work(self):  
        pass  
  
class Eatable(ABC):  
    @abstractmethod  
    def eat(self):  
        pass  
  
class Human(Workable, Eatable):  
    def work(self):  
        return "Working"  
  
    def eat(self):  
        return "Eating"  
  
class Robot(Workable):  
    def work(self):  
        return "Working"
```

Q46. Explain Dependency Inversion Principle (DIP).

Answer:

Principle:

- High-level modules shouldn't depend on low-level modules
- Both should depend on abstractions

Bad example:

```
class MySQLDatabase:  
    def save(self, data):  
        print("Saving to MySQL")  
  
class UserService:  
    def __init__(self):  
        self.db = MySQLDatabase() # Tight coupling  
  
    def save_user(self, user):  
        self.db.save(user)
```

Good example:

```

class Database(ABC):
    @abstractmethod
    def save(self, data):
        pass

class MySQLDatabase(Database):
    def save(self, data):
        print("Saving to MySQL")

class PostgreSQLDatabase(Database):
    def save(self, data):
        print("Saving to PostgreSQL")

class UserService:
    def __init__(self, database: Database):
        self.db = database # Depends on abstraction

    def save_user(self, user):
        self.db.save(user)

# Usage
db = MySQLDatabase()
service = UserService(db)

```

Q47. Explain Association, Aggregation, and Composition.

Answer:

These are **object relationships** describing how classes/objects interact.

1. Association (Weakest relationship):

- General relationship between two independent classes
- Both objects can exist independently
- "uses-a" or "knows-a" relationship

```

class Teacher:
    def __init__(self, name):
        self.name = name

    def teach(self, student):
        print(f"{self.name} teaches {student.name}")

class Student:
    def __init__(self, name):
        self.name = name

    def learn_from(self, teacher):

```

```

        print(f"{self.name} learns from {teacher.name}")

# Association - both can exist independently
teacher = Teacher("Dr. Smith")
student = Student("John")
teacher.teach(student) # They interact but are independent

```

2. Aggregation (HAS-A relationship):

- "Whole-part" relationship where part can exist independently
- Child can exist without parent
- Diamond symbol (hollow) in UML

```

class Professor:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Professor {self.name}"

class Department:
    def __init__(self, name):
        self.name = name
        self.professors = [] # HAS-A professors

    def add_professor(self, professor):
        self.professors.append(professor)

    def list_professors(self):
        return [str(p) for p in self.professors]

# Aggregation - professors exist independently
prof1 = Professor("Alice")
prof2 = Professor("Bob")

cs_dept = Department("Computer Science")
cs_dept.add_professor(prof1)
cs_dept.add_professor(prof2)

# If department is deleted, professors still exist
del cs_dept
print(prof1) # Still exists!

```

3. Composition (Strong HAS-A relationship):

- Child cannot exist without parent
- Lifetime dependency
- Filled diamond in UML
- When parent dies, child dies

```

class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower
        print(f"Engine created: {horsepower}hp")

    def start(self):
        return "Engine started"

    def __del__(self):
        print(f"Engine destroyed")

class Car:
    def __init__(self, model, horsepower):
        self.model = model
        self.engine = Engine(horsepower) # Composition - engine created with car

    def start(self):
        return f"{self.model}: {self.engine.start()}"

    def __del__(self):
        print(f"Car {self.model} destroyed")

# Composition - engine exists only within car
car = Car("Tesla Model S", 1020)
print(car.start())

# When car is destroyed, engine is destroyed too
del car # Destroys both car and engine

```

Comparison:

Aspect	Association	Aggregation	Composition
Relationship	Uses-a	Has-a (weak)	Has-a (strong)
Lifetime	Independent	Independent	Dependent
Example	Teacher-Student	Department-Professor	Car-Engine
UML Symbol	Line	Hollow diamond	Filled diamond

When to use:

- **Association:** Objects interact but are independent
- **Aggregation:** Parent contains children, but children can exist alone
- **Composition:** Parent owns children completely

Q48. Explain Coupling and Cohesion.

Answer:

Coupling: Degree of interdependence between modules/classes.

Types of Coupling (Best to Worst):**1. No Coupling** (Best):

```
class ClassA:
    def method_a(self):
        return "A"

class ClassB:
    def method_b(self):
        return "B"

# No relationship at all
```

2. Loose/Weak Coupling (Good):

```
from abc import ABC, abstractmethod

# Depend on abstraction, not concrete class
class Database(ABC):
    @abstractmethod
    def save(self, data):
        pass

class UserService:
    def __init__(self, database: Database):
        self.db = database # Depends on interface

    def save_user(self, user):
        self.db.save(user)

# Easy to swap implementations
```

3. Tight/Strong Coupling (Bad):

```
class MySQLDatabase:
    def save(self, data):
        print("Saving to MySQL")

class UserService:
    def __init__(self):
        self.db = MySQLDatabase() # Hard-coded dependency

    def save_user(self, user):
        self.db.save(user)

# Can't easily switch to PostgreSQL
```

Cohesion: How closely related and focused the responsibilities of a module/class are.

Types of Cohesion (Best to Worst):

1. High Cohesion (Best):

```
# Single responsibility - only user validation
class UserValidator:
    def validate_email(self, email):
        return '@' in email

    def validate_password(self, password):
        return len(password) >= 8

    def validate_username(self, username):
        return len(username) >= 3

# All methods related to user validation
```

2. Low Cohesion (Bad):

```
# God class - does too many unrelated things
class UserManager:
    def validate_email(self, email):
        pass

    def send_email(self, email, message):
        pass

    def hash_password(self, password):
        pass

    def generate_pdf_report(self):
        pass

    def calculate_taxes(self):
        pass

# Too many unrelated responsibilities!
```

Refactoring for High Cohesion:

```
# Split into focused classes
class UserValidator:
    def validate_email(self, email):
        return '@' in email

    def validate_password(self, password):
```

```

        return len(password) >= 8

class EmailService:
    def send_email(self, to, message):
        print(f"Sending to {to}: {message}")

class PasswordHasher:
    def hash_password(self, password):
        import hashlib
        return hashlib.sha256(password.encode()).hexdigest()

class ReportGenerator:
    def generate_pdf(self, data):
        return f"PDF report: {data}"

# Each class has single, focused responsibility

```

Goal: Low Coupling + High Cohesion

```

# Good design example
class Order:
    """High cohesion - only order-related logic"""
    def __init__(self, items):
        self.items = items

    def total(self):
        return sum(item.price for item in self.items)

class PaymentProcessor:
    """High cohesion - only payment logic"""
    def __init__(self, payment_gateway): # Loose coupling via DI
        self.gateway = payment_gateway

    def process(self, amount):
        return self.gateway.charge(amount)

class OrderService:
    """Loose coupling - depends on abstractions"""
    def __init__(self, payment_processor: PaymentProcessor):
        self.payment_processor = payment_processor

    def checkout(self, order: Order):
        total = order.total()
        return self.payment_processor.process(total)

```

Benefits:

- **Low Coupling:** Easy to change, test, reuse
- **High Cohesion:** Easy to understand, maintain, modify

Q49. Explain Message Passing in OOP.

Answer:

Message Passing: Objects communicate by sending messages (calling methods) to each other.

Key concepts:

- Objects encapsulate data and behavior
- Objects interact through well-defined interfaces
- No direct access to internal state
- Promotes loose coupling

Example - Without Message Passing (Bad):

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance # Public, can be directly modified

# Direct access - breaks encapsulation
account = BankAccount(1000)
account.balance = 0 # Danger! No validation
```

With Message Passing (Good):

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private

    # Messages (methods) to interact
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return f"Deposited ${amount}. Balance: ${self.__balance}"
        return "Invalid amount"

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return f"Withdrew ${amount}. Balance: ${self.__balance}"
        return "Insufficient funds or invalid amount"

    def get_balance(self):
        return self.__balance

# Interact via messages only
account = BankAccount(1000)
print(account.deposit(500)) # Send "deposit" message
print(account.withdraw(200)) # Send "withdraw" message
print(account.get_balance()) # Send "get_balance" message
```

```
# Cannot directly access balance
# print(account.__balance) # Error!
```

Message Passing Between Objects:

```
class Customer:
    def __init__(self, name, account):
        self.name = name
        self.account = account # Reference to account object

    def make_purchase(self, amount):
        # Send message to account object
        result = self.account.withdraw(amount)
        if "Withdrew" in result:
            return f"{self.name} purchased ${amount}"
        return f"{self.name} purchase failed: insufficient funds"

class Notification:
    def send(self, customer, message):
        print(f"Email to {customer.name}: {message}")

class PurchaseService:
    def __init__(self, notifier):
        self.notifier = notifier

    def process_purchase(self, customer, amount):
        # Message passing chain
        result = customer.make_purchase(amount)
        self.notifier.send(customer, result)
        return result

# Usage - Objects communicate via messages
account = BankAccount(1000)
customer = Customer("Alice", account)
notifier = Notification()
service = PurchaseService(notifier)

# Message passing chain:
# service → customer → account → notifier
service.process_purchase(customer, 50)
```

Synchronous vs Asynchronous Message Passing:

Synchronous (Wait for response):

```
class Calculator:
    def add(self, a, b):
        return a + b # Immediate response
```

```
result = Calculator().add(5, 3)
print(result) # 8
```

Asynchronous (Don't wait):

```
import threading

class EmailService:
    def send_async(self, to, message):
        def send():
            print(f"Sending email to {to}...")
            # Simulate delay
            import time
            time.sleep(2)
            print(f"Email sent to {to}")

        thread = threading.Thread(target=send)
        thread.start()
        return "Email queued"

email = EmailService()
result = email.send_async("user@example.com", "Hello")
print(result) # Returns immediately
print("Doing other work...")
```

Benefits:

- **Encapsulation:** Internal state protected
- **Loose coupling:** Objects interact through interfaces
- **Flexibility:** Can change implementation without affecting callers
- **Testability:** Easy to mock message receivers

Q50. Explain Interface vs Abstract Class.

Answer:

Interface: Contract defining what a class should do (method signatures only).

Abstract Class: Partial implementation that can have both abstract and concrete methods.

Python Implementation:

Interface (Pure abstract):

```
from abc import ABC, abstractmethod

class PaymentInterface(ABC):
    """Pure interface - no implementation"""
```

```

    @abstractmethod
    def process_payment(self, amount):
        """All implementing classes must define this"""
        pass

    @abstractmethod
    def refund(self, transaction_id):
        """All implementing classes must define this"""
        pass

# Implementations
class CreditCardPayment(PaymentInterface):
    def process_payment(self, amount):
        return f"Processing ${amount} via Credit Card"

    def refund(self, transaction_id):
        return f"Refunding transaction {transaction_id}"

class PayPalPayment(PaymentInterface):
    def process_payment(self, amount):
        return f"Processing ${amount} via PayPal"

    def refund(self, transaction_id):
        return f"Refunding via PayPal: {transaction_id}"

# Cannot instantiate interface
# payment = PaymentInterface() # Error!

```

Abstract Class (Partial implementation):

```

from abc import ABC, abstractmethod

class Animal(ABC):
    """Abstract class with both abstract and concrete methods"""

    def __init__(self, name):
        self.name = name

    # Concrete method (shared implementation)
    def eat(self):
        return f"{self.name} is eating"

    def sleep(self):
        return f"{self.name} is sleeping"

    # Abstract method (must be implemented by children)
    @abstractmethod
    def speak(self):
        pass

    @abstractmethod

```

```
def move(self):
    pass

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

    def move(self):
        return f"{self.name} runs on four legs"

class Bird(Animal):
    def speak(self):
        return f"{self.name} says Chirp!"

    def move(self):
        return f"{self.name} flies"

# Usage
dog = Dog("Buddy")
print(dog.speak()) # Woof!
print(dog.eat())   # Eating (inherited)
print(dog.move())  # Runs

bird = Bird("Tweety")
print(bird.speak()) # Chirp!
print(bird.move())  # Flies
```

Comparison:

Aspect	Interface	Abstract Class
Methods	Only abstract	Both abstract & concrete
State	No instance variables	Can have instance variables
Constructor	No constructor	Can have constructor
Multiple Inheritance	Can implement multiple	Single inheritance (Python)
Purpose	Define contract	Provide base implementation
When to use	Multiple unrelated classes need same behavior	Related classes share common code

Multiple Interfaces (Python way):

```
class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass
```

```

class Resizable(ABC):
    @abstractmethod
    def resize(self, scale):
        pass

class Movable(ABC):
    @abstractmethod
    def move(self, x, y):
        pass

# Class implementing multiple interfaces
class Shape(Drawable, Resizable, Movable):
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.scale = 1.0

    def draw(self):
        return f"Drawing at ({self.x}, {self.y})"

    def resize(self, scale):
        self.scale = scale
        return f"Resized to {scale}x"

    def move(self, x, y):
        self.x = x
        self.y = y
        return f"Moved to ({x}, {y})"

# Shape must implement all interface methods
shape = Shape(10, 20)
print(shape.draw())
print(shape.resize(2.0))
print(shape.move(30, 40))

```

Real-world example:

```

# Abstract class - shared behavior
class Vehicle(ABC):
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def get_info(self): # Concrete method
        return f"{self.make} {self.model}"

    @abstractmethod
    def start_engine(self): # Abstract
        pass

# Interface - capability contract
class Electric(ABC):

```

```

    @abstractmethod
    def charge(self):
        pass

# Concrete class using both
class Tesla(Vehicle, Electric):
    def start_engine(self):
        return "Tesla starting silently..."

    def charge(self):
        return "Charging battery to 100%"

tesla = Tesla("Tesla", "Model 3")
print(tesla.get_info())      # From abstract class
print(tesla.start_engine())  # Implemented abstract method
print(tesla.charge())        # From interface

```

When to use:

- **Interface:** Define what multiple unrelated classes can do
- **Abstract Class:** Share common code among related classes

Best practices:

- Use interfaces for loose coupling
- Use abstract classes to avoid code duplication
- Favor composition over inheritance
- Keep interfaces small and focused (Interface Segregation Principle)

Quick Interview Tips

1. **Know the 4 pillars:** Encapsulation, Abstraction, Inheritance, Polymorphism
2. **Understand SOLID:** Can explain each with examples
3. **Design patterns:** Know at least 5-7 common ones (Strategy, Decorator, Factory, Singleton, Observer)
4. **Object relationships:** Association, Aggregation, Composition differences
5. **Coupling & Cohesion:** Low coupling + High cohesion = Good design
6. **Message Passing:** Objects communicate through methods, not direct access
7. **Interface vs Abstract Class:** When to use each
8. **Real-world examples:** Relate concepts to practical scenarios
9. **Trade-offs:** Discuss when to use inheritance vs composition
10. **Code examples:** Be ready to write code
11. **Python-specific:** Know Python's OOP features (properties, descriptors, metaclasses, @abstractmethod)

Key OOP Concepts Checklist:

- ☒ Four pillars (Encapsulation, Abstraction, Inheritance, Polymorphism)
- ☒ SOLID principles (all 5)
- ☒ Object relationships (Association, Aggregation, Composition)
- ☒ Coupling and Cohesion

- ☒ Message Passing
- ☒ Interface vs Abstract Class
- ☒ Design patterns (12 patterns covered: Strategy, Decorator, Adapter, Builder, Prototype, Facade, Proxy, Command, State, Template Method, plus DI and MVC)
- ☒ Composition vs Inheritance
- ☒ Magic methods (dunder methods)

Advanced OOP Topics (Questions 51-75)

Q51. Explain descriptors in Python.

Answer:

Descriptors: Objects that define how attributes are accessed (get, set, delete).

Descriptor Protocol:

```
class Descriptor:
    def __get__(self, obj, objtype=None):
        """Called when attribute is accessed"""
        return "Getting value"

    def __set__(self, obj, value):
        """Called when attribute is set"""
        print(f"Setting value to {value}")

    def __delete__(self, obj):
        """Called when attribute is deleted"""
        print("Deleting attribute")

class MyClass:
    attr = Descriptor() # Descriptor attribute

obj = MyClass()
print(obj.attr)        # Calls __get__
obj.attr = 10          # Calls __set__
del obj.attr           # Calls __delete__
```

Practical example - Validation descriptor:

```
class ValidatedString:
    def __init__(self, min_length=0, max_length=100):
        self.min_length = min_length
        self.max_length = max_length
        self.data = {}

    def __set_name__(self, owner, name):
        self.name = name
```

```

def __get__(self, obj, objtype=None):
    if obj is None:
        return self
    return self.data.get(id(obj), "")

def __set__(self, obj, value):
    if not isinstance(value, str):
        raise TypeError(f"{self.name} must be a string")
    if len(value) < self.min_length:
        raise ValueError(f"{self.name} too short (min {self.min_length})")
    if len(value) > self.max_length:
        raise ValueError(f"{self.name} too long (max {self.max_length})")
    self.data[id(obj)] = value

class User:
    username = ValidatedString(min_length=3, max_length=20)
    email = ValidatedString(min_length=5, max_length=100)

    def __init__(self, username, email):
        self.username = username
        self.email = email

# Usage
user = User("john", "john@example.com")
print(user.username) # john

# user.username = "ab" # ValueError: username too short
# user.username = 123 # TypeError: username must be a string

```

Property is a descriptor:

```

# @property is built on descriptors
class Property:
    def __init__(self, fget=None, fset=None, fdel=None):
        self.fget = fget
        self.fset = fset
        self.fdel = fdel

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        if self.fget is None:
            raise AttributeError("unreadable attribute")
        return self.fget(obj)

    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError("can't set attribute")
        self.fset(obj, value)

```

Use cases:

- Data validation
- Lazy loading
- Type checking
- Logging attribute access
- ORM fields

Q52. Explain context managers and `__enter__`/`__exit__`.

Answer:

Context Manager: Manages resources (file, database, locks) using `with` statement.

Protocol methods:

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Called when entering 'with' block"""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Called when exiting 'with' block"""
        if self.file:
            self.file.close()
        # Return False to propagate exceptions
        # Return True to suppress exceptions
        return False

# Usage
with FileManager('test.txt', 'w') as f:
    f.write("Hello, World!")
# File automatically closed after block
```

Database connection example:

```
class DatabaseConnection:
    def __init__(self, connection_string):
        self.connection_string = connection_string
        self.connection = None

    def __enter__(self):
        print("Connecting to database...")
        # self.connection = psycopg2.connect(self.connection_string)
```

```

        self.connection = "Connected"
        return self.connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"Exception occurred: {exc_val}")
            # Rollback transaction
            print("Closing database connection...")
            self.connection = None
            return False # Propagate exception

with DatabaseConnection("postgresql://...") as conn:
    print(f"Using {conn}")
    # Do database operations

```

Using contextlib:

```

from contextlib import contextmanager

@contextmanager
def file_manager(filename, mode):
    """Simpler context manager using decorator"""
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()

with file_manager('test.txt', 'w') as f:
    f.write("Hello!")

```

Multiple context managers:

```

with open('input.txt', 'r') as infile, \
     open('output.txt', 'w') as outfile:
    content = infile.read()
    outfile.write(content.upper())

```

Benefits:

- Automatic resource management
- Exception-safe cleanup
- Cleaner code
- RAII pattern (Resource Acquisition Is Initialization)

Q53. Explain `__call__` and callable objects.

Answer:

`__call__`: Makes object callable like a function.

Basic example:

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

double = Multiplier(2)
triple = Multiplier(3)

print(double(5))    # 10
print(triple(5))    # 15

# Object behaves like a function!
print(callable(double)) # True
```

State-preserving function:

```
class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count

counter = Counter()
print(counter()) # 1
print(counter()) # 2
print(counter()) # 3
```

Decorator class:

```
class LogExecutionTime:
    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        import time
        start = time.time()
        result = self.func(*args, **kwargs)
        end = time.time()
        print(f"{self.func.__name__} took {end - start:.4f} seconds")
        return result
```

```

@LogExecutionTime
def slow_function():
    import time
    time.sleep(1)
    return "Done"

slow_function()
# slow_function took 1.0001 seconds

```

Caching/Memoization:

```

class Memoize:
    def __init__(self, func):
        self.func = func
        self.cache = {}

    def __call__(self, *args):
        if args not in self.cache:
            self.cache[args] = self.func(*args)
        return self.cache[args]

@Memoize
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(100)) # Fast with caching!

```

Use cases:

- Stateful functions
- Decorators
- Callbacks
- Strategy pattern
- Functors

Q54. Explain slots (`__slots__`) for memory optimization.

Answer:

`__slots__`: Restrict attributes, reduce memory usage.

Without slots (uses `__dict__`):

```

class WithoutSlots:
    def __init__(self, x, y):
        self.x = x

```

```

        self.y = y

obj = WithoutSlots(1, 2)
print(obj.__dict__) # {'x': 1, 'y': 2}

# Can add attributes dynamically
obj.z = 3 # Works

```

With slots:

```

class WithSlots:
    __slots__ = ['x', 'y'] # Only these attributes allowed

    def __init__(self, x, y):
        self.x = x
        self.y = y

obj = WithSlots(1, 2)
# print(obj.__dict__) # AttributeError: no __dict__

# Cannot add new attributes
# obj.z = 3 # AttributeError: 'WithSlots' object has no attribute 'z'

```

Memory comparison:

```

import sys

class Normal:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

class Slotted:
    __slots__ = ['x', 'y', 'z']

    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

normal = Normal(1, 2, 3)
slotted = Slotted(1, 2, 3)

print(f"Normal: {sys.getsizeof(normal.__dict__)} bytes")
print(f"Slotted: {sys.getsizeof(slotted)} bytes")
# Slotted uses significantly less memory

```

Inheritance with slots:

```
class Base:
    __slots__ = ['x']

class Child(Base):
    __slots__ = ['y'] # Adds to parent slots

    def __init__(self, x, y):
        self.x = x
        self.y = y

# Child has slots: 'x' and 'y'
```

When to use:

- Creating millions of objects
- Memory-constrained environments
- Known, fixed attributes
- Performance-critical code

Trade-offs:

- ☒ Faster attribute access
- ☒ Reduced memory (~40-50%)
- ☒ No dynamic attributes
- ☒ No `__dict__`
- ☒ Slightly more complex

Q55. Explain operator overloading comprehensively.

Answer:

Operator Overloading: Define custom behavior for operators.

Arithmetic operators:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)
```

```

def __truediv__(self, scalar):
    return Vector(self.x / scalar, self.y / scalar)

def __str__(self):
    return f"Vector({self.x}, {self.y})"

v1 = Vector(3, 4)
v2 = Vector(1, 2)

print(v1 + v2)    # Vector(4, 6)
print(v1 - v2)    # Vector(2, 2)
print(v1 * 3)     # Vector(9, 12)
print(v1 / 2)     # Vector(1.5, 2.0)

```

Comparison operators:

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __eq__(self, other):
        return self.age == other.age

    def __ne__(self, other):
        return self.age != other.age

    def __lt__(self, other):
        return self.age < other.age

    def __le__(self, other):
        return self.age <= other.age

    def __gt__(self, other):
        return self.age > other.age

    def __ge__(self, other):
        return self.age >= other.age

p1 = Person("Alice", 30)
p2 = Person("Bob", 25)

print(p1 > p2)    # True
print(p1 == p2)   # False
print(sorted([p1, p2], key=lambda p: p.age)) # Works!

```

Container operators:

```

class CustomList:
    def __init__(self, items):
        self.items = items

    def __getitem__(self, index):
        return self.items[index]

    def __setitem__(self, index, value):
        self.items[index] = value

    def __len__(self):
        return len(self.items)

    def __contains__(self, item):
        return item in self.items

    def __iter__(self):
        return iter(self.items)

cl = CustomList([1, 2, 3, 4, 5])
print(cl[0])          # 1 (__getitem__)
print(len(cl))        # 5 (__len__)
print(3 in cl)        # True (__contains__)
for item in cl:       # __iter__
    print(item)

```

Unary operators:

```

class Number:
    def __init__(self, value):
        self.value = value

    def __neg__(self):
        return Number(-self.value)

    def __pos__(self):
        return Number(+self.value)

    def __abs__(self):
        return Number(abs(self.value))

    def __invert__(self):
        return Number(~self.value)

n = Number(-5)
print((-n).value)    # 5
print(abs(n).value)  # 5

```

Augmented assignment:

```

class Counter:
    def __init__(self, value):
        self.value = value

    def __iadd__(self, other): # +=
        self.value += other
        return self

    def __isub__(self, other): # -=
        self.value -= other
        return self

c = Counter(10)
c += 5
print(c.value) # 15

```

Complete operator mapping:

Operator	Method	Example
+	<code>__add__</code>	<code>a + b</code>
-	<code>__sub__</code>	<code>a - b</code>
*	<code>__mul__</code>	<code>a * b</code>
/	<code>__truediv__</code>	<code>a / b</code>
//	<code>__floordiv__</code>	<code>a // b</code>
%	<code>__mod__</code>	<code>a % b</code>
**	<code>__pow__</code>	<code>a ** b</code>
==	<code>__eq__</code>	<code>a == b</code>
!=	<code>__ne__</code>	<code>a != b</code>
<	<code>__lt__</code>	<code>a < b</code>
>	<code>__gt__</code>	<code>a > b</code>
<=	<code>__le__</code>	<code>a <= b</code>
>=	<code>__ge__</code>	<code>a >= b</code>
[]	<code>__getitem__</code>	<code>a[key]</code>
in	<code>__contains__</code>	<code>item in a</code>

Q56. Explain the Iterator and Iterable protocols.

Answer:

Iterable: Object that can be looped over (has `__iter__`). **Iterator:** Object that produces values one at a time (has `__iter__` and `__next__`).

Creating iterable:

```
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        """Return iterator object"""
        return CountdownIterator(self.start)

class CountdownIterator:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        """Iterator must return itself"""
        return self

    def __next__(self):
        """Return next value or raise StopIteration"""
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

# Usage
for num in Countdown(5):
    print(num)  # 5, 4, 3, 2, 1
```

Combined iterable and iterator:

```
class Fibonacci:
    def __init__(self, max_count):
        self.max_count = max_count
        self.count = 0
        self.a, self.b = 0, 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.max_count:
            raise StopIteration

        self.count += 1
        result = self.a
        self.a, self.b = self.b, self.a + self.b
```

```

        return result

for num in Fibonacci(10):
    print(num)  # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

```

Using generator (simpler):

```

class FibonacciGenerator:
    def __init__(self, max_count):
        self.max_count = max_count

    def __iter__(self):
        a, b = 0, 1
        for _ in range(self.max_count):
            yield a
            a, b = b, a + b

for num in FibonacciGenerator(10):
    print(num)

```

Custom range:

```

class MyRange:
    def __init__(self, start, end, step=1):
        self.start = start
        self.end = end
        self.step = step

    def __iter__(self):
        current = self.start
        while current < self.end:
            yield current
            current += self.step

for i in MyRange(0, 10, 2):
    print(i)  # 0, 2, 4, 6, 8

```

Reverse iterator:

```

class ReverseIterable:
    def __init__(self, data):
        self.data = data

    def __iter__(self):
        return reversed(self.data)

    def __reversed__(self):

```

```

        """Custom reverse iteration"""
        for item in self.data[::-1]:
            yield item

ri = ReverseIterable([1, 2, 3, 4, 5])
for item in ri:
    print(item)  # 5, 4, 3, 2, 1

```

Q57. Explain generators and yield keyword.

Answer:

Generator: Function that returns iterator using `yield`.

Basic generator:

```

def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

for num in count_up_to(5):
    print(num)  # 1, 2, 3, 4, 5

```

vs Regular function:

```

# Regular function - returns all at once
def get_numbers(n):
    result = []
    for i in range(n):
        result.append(i)
    return result

# Generator - yields one at a time
def get_numbers_gen(n):
    for i in range(n):
        yield i

# Memory efficient for large n!

```

Generator expression:

```

# List comprehension (all in memory)
squares_list = [x**2 for x in range(1000000)]

# Generator expression (lazy evaluation)

```

```
squares_gen = (x**2 for x in range(1000000))

# Only generates values when needed
for square in squares_gen:
    if square > 1000:
        break
```

Infinite generators:

```
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1

gen = infinite_sequence()
print(next(gen)) # 0
print(next(gen)) # 1
print(next(gen)) # 2
```

Pipeline of generators:

```
def read_file(filename):
    with open(filename) as f:
        for line in f:
            yield line.strip()

def filter_comments(lines):
    for line in lines:
        if not line.startswith('#'):
            yield line

def uppercase(lines):
    for line in lines:
        yield line.upper()

# Chain generators
lines = read_file('data.txt')
filtered = filter_comments(lines)
processed = uppercase(filtered)

for line in processed:
    print(line)
```

Generator with send():

```
def echo_generator():
    value = None
    while True:
        value = yield value
        if value is not None:
            value = f"Echo: {value}"

gen = echo_generator()
next(gen) # Prime the generator
print(gen.send("Hello")) # Echo: Hello
print(gen.send("World")) # Echo: World
```

Benefits:

- Memory efficient
- Lazy evaluation
- Can represent infinite sequences
- Clean pipeline syntax

Q58. Explain the difference between `__new__` and `__init__`.

Answer:

`__new__`: Creates the object (called first). `__init__`: Initializes the object (called second).

Normal flow:

```
class Person:
    def __new__(cls, name):
        print(f"1. __new__ called for {cls}")
        instance = super().__new__(cls)
        print(f"2. Instance created: {instance}")
        return instance

    def __init__(self, name):
        print(f"3. __init__ called")
        self.name = name
        print(f"4. Instance initialized: {self.name}")

person = Person("Alice")
# Output:
# 1. __new__ called for <class '__main__.Person'>
# 2. Instance created: <__main__.Person object at 0x...>
# 3. __init__ called
# 4. Instance initialized: Alice
```

Key differences:

Aspect	<code>__new__</code>	<code>__init__</code>
--------	----------------------	-----------------------

Aspect	<code>__new__</code>	<code>__init__</code>
Purpose	Creates object	Initializes object
Called	First	Second
Parameter	cls (class)	self (instance)
Must return	New instance	Nothing (None)
When override	Singleton, immutable types	Most cases

Singleton using `__new__`:

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        print("Init called")

s1 = Singleton() # Init called
s2 = Singleton() # Init called
print(s1 is s2) # True (same object)
```

Customizing immutable types:

```
class PositiveInt(int):
    def __new__(cls, value):
        if value < 0:
            raise ValueError("Value must be positive")
        return super().__new__(cls, value)

# Can't use __init__ for int (immutable)
num = PositiveInt(42)
# num = PositiveInt(-5) # ValueError
```

Factory pattern with `__new__`:

```
class Animal:
    def __new__(cls, animal_type):
        if animal_type == 'dog':
            return Dog()
        elif animal_type == 'cat':
            return Cat()
```

```

        else:
            return super().__new__(cls)

class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

animal = Animal('dog')
print(animal.speak()) # Woof!

```

When to override `__new__`:

- Singleton pattern
- Subclassing immutable types (int, str, tuple)
- Controlling instance creation
- Factory patterns
- Metaclass-like behavior

Q59. Explain weakref and weak references.

Answer:

Weak Reference: Reference that doesn't prevent garbage collection.

Problem without weak references:

```

class Node:
    def __init__(self, value):
        self.value = value
        self.parent = None # Strong reference
        self.children = []

parent = Node("parent")
child = Node("child")
child.parent = parent
parent.children.append(child)

# Circular reference!
# Even if we del parent, child still references it
# Memory leak!

```

Solution with weakref:

```

import weakref

```

```

class Node:
    def __init__(self, value):
        self.value = value
        self._parent = None # Will store weak reference
        self.children = []

    @property
    def parent(self):
        return self._parent() if self._parent else None

    @parent.setter
    def parent(self, node):
        self._parent = weakref.ref(node) if node else None

parent = Node("parent")
child = Node("child")
child.parent = parent
parent.children.append(child)

# Now when we delete parent, it can be garbage collected
del parent
print(child.parent) # None (parent was collected)

```

WeakValueDictionary (cache example):

```

import weakref

class ObjectCache:
    def __init__(self):
        self.cache = weakref.WeakValueDictionary()

    def get_or_create(self, key, factory):
        if key in self.cache:
            print(f"Cache hit: {key}")
            return self.cache[key]

        print(f"Cache miss: {key}")
        obj = factory()
        self.cache[key] = obj
        return obj

class ExpensiveObject:
    def __init__(self, data):
        self.data = data

cache = ObjectCache()

# First call - creates object
obj1 = cache.get_or_create('key1', lambda: ExpensiveObject("data"))

# Second call - returns cached
obj2 = cache.get_or_create('key1', lambda: ExpensiveObject("data"))

```

```
print(obj1 is obj2) # True

# Delete strong reference
del obj1, obj2

# Object garbage collected, cache entry removed automatically
obj3 = cache.get_or_create('key1', lambda: ExpensiveObject("data"))
# Cache miss: key1 (object was collected)
```

Weak callbacks:

```
import weakref

def callback(weak_ref):
    print("Object is being destroyed!")

class MyClass:
    pass

obj = MyClass()
weak = weakref.ref(obj, callback)

print(weak()) # <__main__.MyClass object>
del obj      # Object is being destroyed!
print(weak()) # None
```

Use cases:

- Caching
- Observer pattern (avoid memory leaks)
- Parent-child relationships
- Breaking circular references

Q60. Explain the concept of mixins in detail.

Answer:

Mixin: Small class that adds specific functionality, designed to be inherited alongside other classes.

Characteristics:

- Not meant to stand alone
- Provides specific, focused functionality
- Multiple mixins can be combined
- Alternative to deep inheritance

Example - Logging mixin:

```

class LoggerMixin:
    def log(self, message, level="INFO"):
        print(f"[{level}] {self.__class__.__name__}: {message}")

class TimestampMixin:
    def get_timestamp(self):
        from datetime import datetime
        return datetime.now().isoformat()

class SerializableMixin:
    def to_dict(self):
        return {k: v for k, v in self.__dict__.items()
                if not k.startswith('_')}

class User(LoggerMixin, TimestampMixin, SerializableMixin):
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def save(self):
        self.log(f"Saving user at {self.get_timestamp()}")
        # Save logic
        return self.to_dict()

user = User("Alice", "alice@example.com")
user.log("User created")
print(user.get_timestamp())
print(user.to_dict())

```

JSON mixin:

```

import json

class JSONMixin:
    def to_json(self):
        return json.dumps(self.to_dict())

    @classmethod
    def from_json(cls, json_string):
        data = json.loads(json_string)
        return cls(**data)

    def to_dict(self):
        return self.__dict__

class Product(JSONMixin):
    def __init__(self, name, price):
        self.name = name
        self.price = price

product = Product("Laptop", 999.99)

```

```

json_str = product.to_json()
print(json_str)  # {"name": "Laptop", "price": 999.99}

restored = Product.from_json(json_str)
print(restored.name)  # Laptop

```

Comparison mixin:

```

class ComparableMixin:
    """Implements all comparison operators based on _cmp_key"""

    def _cmp_key(self):
        raise NotImplementedError("Subclass must implement _cmp_key")

    def __eq__(self, other):
        return self._cmp_key() == other._cmp_key()

    def __ne__(self, other):
        return not self.__eq__(other)

    def __lt__(self, other):
        return self._cmp_key() < other._cmp_key()

    def __le__(self, other):
        return self._cmp_key() <= other._cmp_key()

    def __gt__(self, other):
        return self._cmp_key() > other._cmp_key()

    def __ge__(self, other):
        return self._cmp_key() >= other._cmp_key()

class Person(ComparableMixin):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def _cmp_key(self):
        return self.age

p1 = Person("Alice", 30)
p2 = Person("Bob", 25)
print(p1 > p2)  # True
print(sorted([p1, p2], key=lambda p: p))  # Sorted by age

```

Best practices:

- Keep mixins focused (single responsibility)
- Name with "Mixin" suffix
- Don't use `__init__` in mixins (or call `super()`)

- Document dependencies
- Order matters in multiple inheritance

vs Inheritance:

- Mixin: Adds specific features (horizontal extension)
- Inheritance: IS-A relationship (vertical extension)

vs Composition:

- Mixin: Compile-time, uses inheritance
- Composition: Runtime, uses delegation

Q61. Explain monkey patching and its implications.

Answer:

Monkey Patching: Modifying or extending code at runtime.

Basic example:

```
class MyClass:
    def original_method(self):
        return "Original"

# Monkey patch: add new method
def new_method(self):
    return "New"

MyClass.new_method = new_method

obj = MyClass()
print(obj.original_method()) # Original
print(obj.new_method())     # New
```

Replacing existing method:

```
class Calculator:
    def add(self, a, b):
        return a + b

# Monkey patch: modify behavior
original_add = Calculator.add

def add_with_logging(self, a, b):
    print(f"Adding {a} + {b}")
    return original_add(self, a, b)

Calculator.add = add_with_logging
```

```
calc = Calculator()
print(calc.add(5, 3))
# Adding 5 + 3
# 8
```

Patching built-in types (dangerous!):

```
# Don't do this in production!
def reverse_string(self):
    return self[::-1]

str.reverse = reverse_string

text = "hello"
print(text.reverse()) # olleh
```

Testing with monkey patching:

```
import datetime

class TimeDependent:
    def get_message(self):
        now = datetime.datetime.now()
        if now.hour < 12:
            return "Good morning"
        return "Good afternoon"

# Monkey patch for testing
def fake_now():
    return datetime.datetime(2024, 1, 1, 9, 0) # 9 AM

original_now = datetime.datetime.now
datetime.datetime.now = staticmethod(fake_now)

obj = TimeDependent()
print(obj.get_message()) # Good morning (always)

# Restore
datetime.datetime.now = original_now
```

Using unittest.mock (better approach):

```
from unittest.mock import patch
import datetime

class TimeDependent:
    def get_message(self):
```

```
    now = datetime.datetime.now()
    if now.hour < 12:
        return "Good morning"
    return "Good afternoon"

# Mock with context manager
with patch('datetime.datetime') as mock_dt:
    mock_dt.now.return_value = datetime.datetime(2024, 1, 1, 9, 0)
    obj = TimeDependent()
    print(obj.get_message()) # Good morning
```

Pros:

- Quick fixes/workarounds
- Testing
- Hot-patching bugs
- Extending third-party code

Cons:

- Hard to maintain
- Breaks expectations
- Can cause subtle bugs
- Makes code harder to understand
- Version compatibility issues

Best practices:

- Use only for testing
- Document extensively
- Restore original after use
- Consider alternatives (inheritance, composition, decorators)
- Use mocking libraries (unittest.mock, pytest)

Q62. Explain the Law of Demeter (Principle of Least Knowledge).

Answer:

Law of Demeter: Object should only talk to its immediate friends, not strangers.

"Don't talk to strangers":

- Only call methods on:
 1. The object itself (self)
 2. Objects passed as parameters
 3. Objects created locally
 4. Instance variables

Violation example:

```
class Address:
    def __init__(self, city):
        self.city = city

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address

class Order:
    def __init__(self, person):
        self.person = person

    def get_shipping_city(self):
        # Violation! Reaching through multiple objects
        return self.person.address.city

# Usage
address = Address("New York")
person = Person("Alice", address)
order = Order(person)
print(order.get_shipping_city()) # Too much knowledge!
```

Following Law of Demeter:

```
class Address:
    def __init__(self, city):
        self.city = city

    def get_city(self):
        return self.city

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address

    def get_city(self):
        return self.address.get_city()

class Order:
    def __init__(self, person):
        self.person = person

    def get_shipping_city(self):
        # Better! Only talk to immediate friend
        return self.person.get_city()

# Usage
address = Address("New York")
person = Person("Alice", address)
```

```
order = Order(person)
print(order.get_shipping_city())
```

Real-world example - Shopping cart:

```
# Bad - violates Law of Demeter
class ShoppingCart:
    def get_total_price(self):
        total = 0
        for item in self.items:
            total += item.product.get_price() * item.quantity
        return total

# Good - follows Law of Demeter
class CartItem:
    def __init__(self, product, quantity):
        self.product = product
        self.quantity = quantity

    def get_total_price(self):
        return self.product.get_price() * self.quantity

class ShoppingCart:
    def get_total_price(self):
        return sum(item.get_total_price() for item in self.items)
```

Benefits:

- Loose coupling
- Easier to change implementation
- More maintainable
- Clearer responsibilities
- Better encapsulation

Trade-offs:

- More wrapper methods
- Can feel verbose
- Sometimes creates "middle-man" classes

Q63. Explain GRASP principles (General Responsibility Assignment Software Patterns).

Answer:

GRASP: Nine fundamental principles for object-oriented design.

1. Information Expert: Assign responsibility to class with most information needed.

```
class Order:
    def __init__(self, items):
        self.items = items

    def get_total(self):
        # Order has the information (items), so it calculates total
        return sum(item.price * item.quantity for item in self.items)
```

2. Creator: Class B should create class A if B contains/aggregates/records A.

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_product(self, product, quantity):
        # Cart creates CartItem because it contains items
        item = CartItem(product, quantity)
        self.items.append(item)
        return item
```

3. Controller: First object beyond UI that handles system event.

```
class OrderController:
    def __init__(self, order_service):
        self.order_service = order_service

    def create_order(self, customer_id, items):
        # Controller coordinates, doesn't do business logic
        customer = self.customer_service.get(customer_id)
        order = self.order_service.create(customer, items)
        return order
```

4. Low Coupling: Minimize dependencies between classes.

```
# Bad - high coupling
class EmailSender:
    def send(self, message):
        # Directly coupled to SMTP
        import smtplib
        # SMTP logic...

# Good - low coupling
class EmailSender:
    def __init__(self, email_service):
        self.email_service = email_service # Dependency injection
```

```
def send(self, message):
    self.email_service.send(message)
```

5. High Cohesion: Keep related responsibilities together.

```
# Good - high cohesion
class UserValidator:
    def validate_email(self, email):
        pass

    def validate_password(self, password):
        pass

    def validate_username(self, username):
        pass

# All methods related to user validation
```

6. Polymorphism: Use polymorphism instead of conditional logic.

```
# Bad
class PaymentProcessor:
    def process(self, payment_type, amount):
        if payment_type == "credit_card":
            # Credit card logic
            pass
        elif payment_type == "paypal":
            # PayPal logic
            pass

# Good
class PaymentProcessor:
    def process(self, payment_method, amount):
        payment_method.pay(amount) # Polymorphism!
```

7. Pure Fabrication: Create artificial class when no domain class fits.

```
# PersistenceManager is pure fabrication
# (not a domain concept, but needed for technical reasons)
class PersistenceManager:
    def save(self, entity):
        # Database logic
        pass

    def load(self, entity_id):
        # Database logic
        pass
```

8. Indirection: Use intermediary to decouple components.

```
# Adapter is indirection
class LegacySystemAdapter:
    def __init__(self, legacy_system):
        self.legacy = legacy_system

    def modern_method(self):
        # Translate to legacy format
        return self.legacy.old_method()
```

9. Protected Variations: Protect from variations using stable interfaces.

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass

# Implementation can change without affecting clients
class StripeGateway(PaymentGateway):
    def process_payment(self, amount):
        # Stripe logic
        pass

class PayPalGateway(PaymentGateway):
    def process_payment(self, amount):
        # PayPal logic
        pass
```

Q64. Explain composition over inheritance in depth.

Answer:

Composition Over Inheritance: Prefer "has-a" over "is-a" relationships.

Problem with inheritance:

```
# Rigid inheritance hierarchy
class Animal:
    def eat(self):
        return "eating"

    def sleep(self):
        return "sleeping"
```

```
class FlyingAnimal(Animal):
    def fly(self):
        return "flying"

class SwimmingAnimal(Animal):
    def swim(self):
        return "swimming"

# What about duck? It flies AND swims!
# Multiple inheritance gets messy
class Duck(FlyingAnimal, SwimmingAnimal):
    pass

# What about penguin? It swims but doesn't fly!
# Inheriting fly() when you don't need it
```

Solution with composition:

```
# Behaviors as separate classes
class FlyBehavior:
    def fly(self):
        return "flying through the air"

class NoFly:
    def fly(self):
        return "can't fly"

class SwimBehavior:
    def swim(self):
        return "swimming in water"

class NoSwim:
    def swim(self):
        return "can't swim"

# Animal composed of behaviors
class Animal:
    def __init__(self, fly_behavior, swim_behavior):
        self.fly_behavior = fly_behavior
        self.swim_behavior = swim_behavior

    def perform_fly(self):
        return self.fly_behavior.fly()

    def perform_swim(self):
        return self.swim_behavior.swim()

# Easy to create any combination!
duck = Animal(FlyBehavior(), SwimBehavior())
print(duck.perform_fly()) # flying through the air
print(duck.perform_swim()) # swimming in water
```

```
penguin = Animal(NoFly(), SwimBehavior())
print(penguin.perform_fly()) # can't fly
print(penguin.perform_swim()) # swimming in water

# Can even change behavior at runtime!
duck.fly_behavior = NoFly()
print(duck.perform_fly()) # can't fly
```

Car example:

```
# Bad inheritance
class Car(Engine, Transmission, Radio, GPS):
    pass # Rigid, hard to modify

# Good composition
class Engine:
    def start(self):
        return "Engine started"

class Transmission:
    def shift(self, gear):
        return f"Shifted to gear {gear}"

class Car:
    def __init__(self):
        self.engine = Engine()
        self.transmission = Transmission()

    def start(self):
        return self.engine.start()

    def drive(self):
        self.start()
        return self.transmission.shift(1)

# Easy to swap components
class ElectricEngine:
    def start(self):
        return "Electric motor engaged"

car = Car()
car.engine = ElectricEngine() # Changed engine!
print(car.start()) # Electric motor engaged
```

When to use each:

Use Inheritance:

- True IS-A relationship
- Liskov Substitution Principle holds

- Polymorphism needed
- Shared interface important

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    # Circle IS-A Shape
    pass
```

Use Composition:

- HAS-A relationship
- Need flexibility
- Want to change behavior at runtime
- Avoid fragile base class problem

```
class Logger:
    def __init__(self, formatter, output):
        self.formatter = formatter # HAS-A formatter
        self.output = output       # HAS-A output
```

Benefits of composition:

- More flexible
- Can change at runtime
- Easier to test (inject dependencies)
- Avoids inheritance problems
- Follows SOLID principles better

Q65. Explain the concept of object identity, equality, and hashing.

Answer:

Three concepts: `is` (identity), `==` (equality), `hash()` (hashability).

Identity (is): Same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is b) # False (different objects)
print(a is c) # True (same object)
print(id(a), id(b), id(c)) # Different IDs for a and b
```

Equality (==): Same value.

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)  # True (same value)
print(a is b)  # False (different objects)
```

Custom equality:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __eq__(self, other):
        if not isinstance(other, Person):
            return False
        return self.name == other.name and self.age == other.age

    def __ne__(self, other):
        return not self.__eq__(other)

p1 = Person("Alice", 30)
p2 = Person("Alice", 30)
p3 = Person("Bob", 25)

print(p1 == p2)  # True (same values)
print(p1 is p2)  # False (different objects)
print(p1 == p3)  # False (different values)
```

Hashing (__hash__): For use in sets and dict keys.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __hash__(self):
        return hash((self.x, self.y))

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)
```

```
# Can use in set
points = {p1, p2, p3}
print(len(points)) # 2 (p1 and p2 are equal)

# Can use as dict key
point_data = {p1: "origin", p3: "other"}
print(point_data[p2]) # "origin" (p2 == p1)
```

Hash contract: If `a == b`, then `hash(a) == hash(b)`

```
class BadHash:
    def __init__(self, value):
        self.value = value

    def __eq__(self, other):
        return self.value == other.value

# BAD: hash not consistent with __eq__
def __hash__(self):
    import random
    return random.randint(0, 1000)

# This will cause problems in sets/dicts!
```

Immutable objects and hashing:

```
# Mutable objects shouldn't be hashable
class MutablePoint:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # Don't implement __hash__ for mutable objects!
    __hash__ = None # Explicitly unhashable

# mp = MutablePoint(1, 2)
# hash(mp) # TypeError: unhashable type

# Immutable objects can be hashable
class ImmutablePoint:
    def __init__(self, x, y):
        self._x = x
        self._y = y

    @property
    def x(self):
        return self._x

    @property
```

```

def y(self):
    return self._y

def __eq__(self, other):
    return self._x == other._x and self._y == other._y

def __hash__(self):
    return hash((self._x, self._y))

ip = ImmutablePoint(1, 2)
print(hash(ip)) # Works!

```

Best practices:

- Only hash immutable objects
- `__eq__` and `__hash__` must be consistent
- If you override `__eq__`, override `__hash__` too
- Use `@dataclass(frozen=True)` for immutable dataclasses

Q66-Q70. [Advanced patterns and anti-patterns to be added...]

Common OOP Anti-Patterns and Code Smells

God Object / God Class

Problem: One class does everything.

```

# Bad
class Application:
    def connect_to_database(self): pass
    def send_email(self): pass
    def process_payment(self): pass
    def generate_report(self): pass
    def validate_user(self): pass
    # 50 more methods...

```

Solution: Split into focused classes following Single Responsibility.

Circular Dependencies

Problem: A depends on B, B depends on A. **Solution:** Introduce interface/abstraction, or restructure.

Leaky Abstraction

Problem: Implementation details leak through interface. **Solution:** Hide internal details completely.

Yo-Yo Problem

Problem: Deep inheritance hierarchies hard to follow. **Solution:** Favor composition, keep hierarchies shallow.

Good luck with your OOP interview! 🚀